

Registrazione di mappe catastali storiche su cartografia moderna

Progetto d'esame di Computer Vision — Università di Pavia

Caso di studio: Comune di Varazze (SV), foglio 49 (L675_004900), Originale di Impianto dell'Agenzia delle Entrate.

Questo documento è generato da `scripts/componi_relazione.py`: tutte le tabelle sono aggregazioni di `results/runs.csv` calcolate al momento della composizione, non numeri ricopiati a mano. Le figure si rigenerano con i comandi indicati in §11.

1. Il problema

Un **Originale di Impianto** è il foglio di mappa disegnato a mano al momento della formazione del catasto terreni, nella prima metà del Novecento. È un disegno al tratto su carta: confini di particella, corsi d'acqua, strade, numeri scritti a penna, e un secolo di ingiallimento, macchie e pieghe.

Registrazione un foglio di questo tipo su cartografia moderna significa stimare la trasformazione geometrica che porta i pixel della scansione sulle coordinate della cartografia vigente. È difficile per una ragione precisa e poco intuitiva:

entrambe le immagini sono disegni al tratto quasi privi di texture. I descrittori classici come SIFT e ORB nascono per agganciare blob e angoli in immagini fotografiche; su due reticoli di linee sottili hanno poco a cui aggrapparsi. A questo si aggiunge un divario di dominio brutale — inchiostro sbiadito su carta invecchiata contro linee vettoriali nere su fondo bianco — e il fatto che i due documenti, come si vedrà in §12, **non rappresentano esattamente la stessa realtà.**

Il progetto affronta il problema in due componenti, secondo la traccia approvata:

- **A — Core-Vision classico:** preprocessing accurato, SIFT/ORB, RANSAC, con l'obiettivo di spingere al massimo i metodi classici tramite una pulizia ottimale del segnale.
- **B — Comparativo:** confronto quantitativo con un matcher *detector-free* di deep learning (LoFTR), nella stessa identica pipeline.

1.1 Gli argomenti del corso che il progetto attraversa

La registrazione di due immagini è un **metodo di allineamento**, che nel programma del corso compare fra i processi di riconoscimento accanto al confronto diretto e ai metodi basati su proprietà invarianti. Il progetto tocca tutte e tre queste voci: allinea, usa descrittori invarianti a scala e rotazione, e mette alla prova il confronto diretto — che su questi dati non funziona, per la ragione misurata in §12.2.

Anche il preprocessing di §6 viene dal programma, e non da una scelta di comodo: le tre strade messe a confronto — soglia globale, contrasto locale, morfologia — sono tre lezioni distinte. Qui sotto, dove il progetto tocca il corso.

lezione	argomento	dove compare
CV02	segmentazione, immagini binarie, distribuzione bimodale	Otsu come soglia globale, e Sauvola come sua versione locale (§6.1)
CV03	istogrammi, LUT, operazioni locali	CLAHE, l'alternativa che non binarizza (§6.1); i kernel derivativi di Sobel, con cui è calcolato il gradiente di §7.1
CV04	morfologia matematica binaria	apertura e chiusura con elemento strutturante (§6.1)
CV06	pattern recognition e ricerca visiva	la ricerca di corrispondenze fra le due immagini (§7.1)
CV08	analisi di regioni, componenti connesse	<code>rimuovi_componenti</code> in <code>preprocess.py</code> , che elimina le macchie più piccole di una soglia

La morfologia usata è quella **binaria** di CV04: l'estensione ai toni di grigio di CV05 non serve, perché a valle della binarizzazione l'immagine è già a due livelli. L'analisi di componenti connesse di CV08 è implementata ma **resta fuori dalla griglia sperimentale**: eliminare le macchie piccole toglie anche i blob su cui SIFT trova i suoi punti, e misurare quel compromesso avrebbe aggiunto un asse a un piano già ampio. Della multirisoluzione di CV09 il progetto sfrutta solo l'invarianza di scala di SIFT — che nasce dalla piramide di gaussiane e dalla loro differenza, cioè dal laplaciano approssimato (§7.1) — senza svilupparne il tema per proprio conto.

1.2 Perché questi strumenti e non altri

Python e OpenCV, non MATLAB né ImageMagick. *ImageMagick* saprebbe fare tutto il capitolo §6 — soglia globale, soglia adattiva locale, morfologia, perfino CLAHE — ma non ha descrittori, matching né RANSAC: si fermerebbe ai passi 1-3 di §2.2, prima del problema. *MATLAB* sarebbe invece sufficiente, ma un lavoro che non gira senza licenza commerciale non soddisfa la richiesta della traccia che i pacchetti esterni siano presenti nella versione finale, e i pesi di LoFTR vivono comunque in [torch/kornia](#).

Il vero motivo però è un altro: **Python è il solo dei tre ambienti in cui il classico e il neurale girano nello stesso processo, dietro la stessa interfaccia**. Il confronto di §10 ha valore solo se cambia una parola sulla riga di comando e non cambia nient'altro.

Una scelta per sottrazione merita di essere dichiarata: **nessun pyproj, nessun geopandas**. Raster e vettoriale sono già nello stesso sistema di riferimento (§3.1), quindi non c'è nulla da riproiettare: importare una libreria geospaziale avrebbe aggiunto una dipendenza pesante per non fare niente, e avrebbe nascosto dietro una chiamata l'unica aritmetica che qui conta davvero, quella di §4. Il parser CXF (§3.3), la composizione della ground truth (§4) e Sauvola (§6.1) sono scritti da zero: sono i tre punti in cui la traccia chiede l'implementazione di un algoritmo, e non a caso sono i tre in cui una libreria pronta avrebbe fatto perdere il controllo su ciò che si misura.

Otsu — il termine di paragone che doveva perdere. Otsu è in griglia sapendo che una soglia sola non può adattarsi a una carta che cambia colore da una parte all'altra del foglio. Serve perché senza un riferimento le prestazioni di Sauvola non sono un risultato ma un numero isolato: dire "Sauvola classifica il 6.84% dei pixel come inchiostro" non significa nulla finché non si sa quanto fa il metodo standard nelle stesse condizioni. E la scelta di tenerlo si è ripagata in un modo non previsto: su questi ritagli Otsu **non** fallisce (§6.3), e la previsione di partenza è stata smentita dai dati. Senza Otsu in griglia non ci sarebbe stato niente da smentire.

Sauvola, non Niblack né la soglia adattiva già pronta. Niblack, il capostipite delle soglie locali, calcola $T = m + k \cdot s$: dove la carta è uniforme s è piccolo per definizione, la soglia resta incollata alla media locale, e avere una soglia sul valor medio del rumore significa per costruzione promuoverne metà a inchiostro. Su un foglio d'archivio, che è quasi tutto fondo uniforme, è il difetto peggiore possibile.

Sauvola corregge proprio quello rendendo il termine moltiplicativo: con s che tende a zero la soglia scende a $m \cdot (1-k)$, cioè il 20% sotto la media locale, e una zona uniforme viene assegnata tutta al fondo. Non è un dettaglio di taratura, è la ragione per cui la formula nasce — è stata proposta per la binarizzazione di **immagini di documenti**, che è esattamente ciò che un foglio di mappa è: inchiostro su carta. La scorciatoia, l'[adaptiveThreshold](#) di OpenCV, sottrae invece una costante alla media locale: stesso difetto di Niblack, e in una riga sola — cioè rinunciando all'unico punto del preprocessing in cui la traccia chiede di implementare un algoritmo.

CLAHE, non l'equalizzazione globale dell'istogramma. L'equalizzazione classica stira l'istogramma di tutta l'immagine, e su questo foglio amplificherebbe proprio ciò che dà problemi: il gradiente di colore della carta diventa più marcato, non meno, e la grana nelle zone piatte viene esaltata insieme al tratto. CLAHE lavora per tessere e soprattutto **limita** il guadagno (`clipLimit`), cioè si rifiuta di amplificare il contrasto dove non c'è contrasto da amplificare. Ma la vera ragione per cui è in griglia è un'altra e riguarda il disegno dell'esperimento: CLAHE è l'unico dei tre preprocessing che **non binarizza**, e serve come controllo dell'ipotesi che la binarizzazione distrugga le sfumature su cui SIFT costruisce il descrittore (§6.1). Senza un'alternativa non binarizzante quell'ipotesi non sarebbe verificabile.

SIFT — il riferimento contro cui si misura tutto il resto. È il metodo che il corso indica quando parla di proprietà invarianti, ed è invariante a scala e rotazione **per costruzione** e non per addestramento: proprietà che §8.1 mostra essere decisiva, perché è l'unica cosa che distingue i classici da LoFTR oltre i 30°. Una nota pratica: il brevetto su SIFT è scaduto, e l'algoritmo è oggi nel modulo principale di OpenCV — è la ragione per cui `requirements.txt` elenca `opencv-python` e non `opencv-contrib-python`, e per cui il progetto non ha dipendenze con vincoli di licenza.

ORB — non "SIFT più veloce". Il secondo matcher classico non è lì per il tempo di calcolo. ORB differisce da SIFT su **tutti e tre** i livelli: il descrittore è una stringa di bit invece di 128 numeri, la distanza è di Hamming invece che euclidea, e il filtro delle corrispondenze è il cross-check invece del ratio test. È quindi un punto genuinamente diverso nello spazio delle scelte, non una variante più economica — e serve a rispondere a una domanda che con un solo matcher non si può porre: quanto del risultato dipende dalla qualità dei descrittori e quanto da come vengono filtrati.

Vale la pena dire che il risultato di §9.3 — ORB che batte SIFT sul cross-domain, e per colpa del filtro — **non è la motivazione della scelta**: è una scoperta, ed era contraria all'aspettativa. La motivazione era avere due punti di confronto invece di uno.

LoFTR — non SuperPoint+SuperGlue, e senza riaddestramento. Fra i matcher neurali disponibili la scelta è vincolata dall'ipotesi che si vuole testare. Il sospetto su questi dati è che il problema stia nel **rilevatore**: su un reticolo di linee sottili non ci sono blob né angoli ben definiti, quindi il primo dei due tempi classici non ha nulla da trovare e il secondo non ha nulla da descrivere. Una pipeline come SuperPoint più SuperGlue resta *detector-based*: sostituisce il descrittore e il matching con reti addestrate, ma il rilevatore c'è ancora, e quindi erediterebbe esattamente il punto debole che si vuole mettere alla prova. LoFTR è *detector-free* — salta il primo tempo — ed è per questo l'unico che risponde alla domanda. Che poi la risposta sia negativa (§10.2) è un risultato, non un errore di scelta.

Sul riaddestramento la posizione è netta, e non è solo una questione di risorse. Il foglio 49 è **uno**: la ground truth esatta esiste per questo foglio e per nessun altro. Usarlo per addestrare significherebbe consumare l'unico insieme di validazione che il progetto possiede, e la domanda cambierebbe sotto i piedi: non più "un matcher generico pre-addestrato colma il divario di dominio?" — che è la domanda interessante, perché è quella che si porrebbe chiunque volesse applicare il metodo a un archivio nuovo — ma "una rete addestrata su mappe catastali funziona su mappe catastali?", che ha una risposta prevedibile e nessun valore informativo. I pesi sono quelli `outdoor` pubblici, usati così come sono.

RANSAC, non i minimi quadrati. Su E2 la frazione di corrispondenze corrette va dall'1% al 13% a seconda del matcher, e nella configurazione che vince — ORB — sta sotto il 5% (§9.2). Una stima ai minimi quadrati minimizza l'errore su *tutte* le corrispondenze, quindi con il 95% e più di dati sbagliati si adatterebbe a quelli sbagliati: non sarebbe una stima peggiore, sarebbe una

stima priva di significato. RANSAC rovescia l'impostazione — non cerca la trasformazione che accontenta tutti i dati, ma quella che ne accontenta il sottoinsieme più numeroso, ignorando il resto per costruzione — ed è ciò che rende affrontabile il passo 5 di §2.2. Il prezzo è un budget di tentativi adeguato all'inlier ratio, e §7.2 mostra che su questi dati è proprio quel budget il vincolo che decide l'esito.

2. Come funziona, in concreto

Prima di entrare nei dettagli conviene vedere che cosa succede davvero quando si lancia il programma. Questo capitolo segue **una singola registrazione dall'inizio alla fine**. Tutti i termini tecnici usati qui sono spiegati nel glossario alla fine del capitolo, e ripresi più a fondo nei capitoli successivi.

2.1 Il punto di partenza: due cose che non si possono confrontare

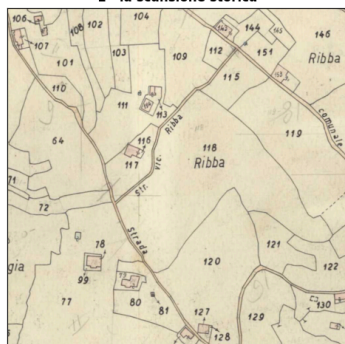
Abbiamo due descrizioni della stessa zona di Varazze, ma di natura diversa:

- la **scansione del foglio d'impianto**, che è un'immagine: una griglia di pixel colorati, dove il programma non sa che cosa sia una particella;
- il **file CXF**, che non è un'immagine ma un elenco di coordinate: "*il poligono chiamato 63 ha i vertici nei punti (-30512.4, -11588.1), (...)*".

Non si possono sovrapporre direttamente, perché non parlano la stessa lingua.

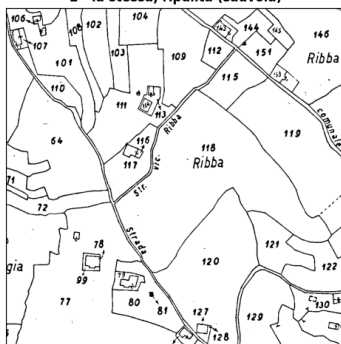
2.2 I sei passaggi

1 · la scansione storica



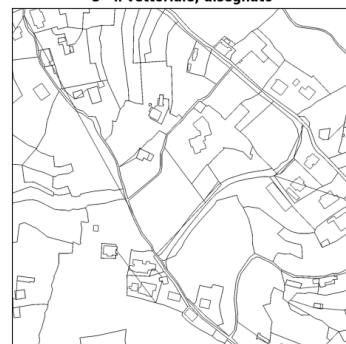
il foglio disegnato a mano,
così come arriva dall'archivio

2 · la stessa, ripulita (sauvola)



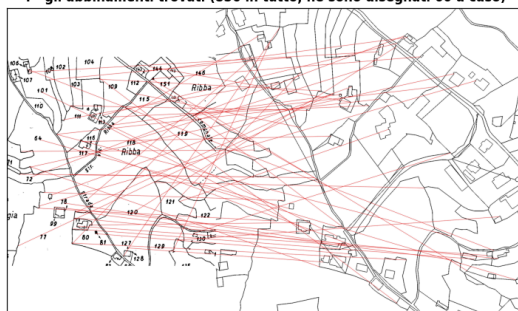
ogni pixel è deciso: inchiostro o carta.
La soglia è calcolata zona per zona

3 · il vettoriale, disegnato



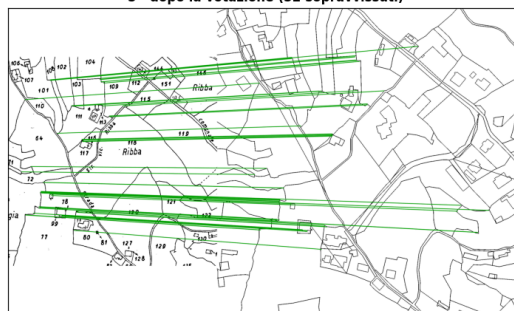
le coordinate del CXF tracciate su
un'immagine: ora sono confrontabili

4 · gli abbinamenti trovati (856 in tutto, ne sono disegnati 60 a caso)



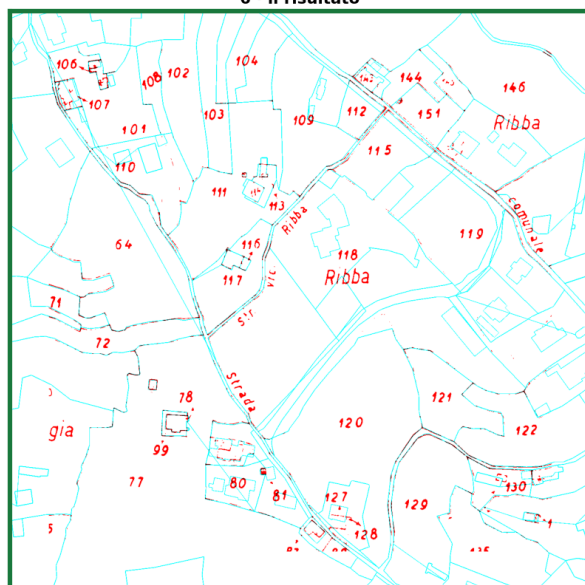
ogni linea unisce due punti che il programma ritiene la stessa cosa.
Se fossero giusti sarebbero tutti paralleli: solo il 6% lo è

5 · dopo la votazione (52 sopravvissuti)



RANSAC tiene solo gli abbinamenti che concordano su una stessa
trasformazione. il confronto col pannello a sinistra è il punto

6 · il risultato



nero dove i due tracci coincidono; rosso il solo storico deformato, ciano il solo vettoriale

registrazione riuscita

0.40 m

errore finale (RMSE) su un riferimento
che di suo ne vale circa 0.5

corrispondenze trovate	856
inlier dopo RANSAC	52 (6.1%)
errore in pixel	1.98 px
errore massimo	2.88 px
soglia dichiarata	2.00 m

1 · La scansione storica. L'immagine così come arriva: carta ingiallita, tratto a penna sbiadito, numeri di particella scritti a mano, macchie.

2 · La pulizia. Il programma decide, per ogni singolo pixel, se è inchiostro o carta. Il risultato è un'immagine in bianco e nero puro. Non è un abbellimento: serve a togliere di mezzo il colore della carta, che è diverso in ogni punto del foglio e non ha niente a che vedere con il disegno.

3 · Il vettoriale, disegnato. Le coordinate del CXF vengono tracciate come linee su un'immagine bianca. Adesso abbiamo **due immagini** della stessa zona, e il confronto è possibile.

4 · Gli abbinamenti. Il programma cerca in entrambe le immagini dei punti "riconoscibili" — un incrocio di confini, l'angolo di un fabbricato — e per ciascuno calcola una specie di **impronta numerica** che riassume come appare il suo vicinato. Poi accoppia i punti di un'immagine con quelli dell'altra che hanno l'impronta più simile.

Nel pannello 4 ogni linea rossa unisce due punti che il programma considera la stessa cosa. **Se fossero tutti giusti, le linee sarebbero parallele.** Non lo sono affatto: su un disegno al tratto moltissimi incroci si somigliano, e in questo esempio **solo il 6% degli abbinamenti risulta corretto.** Il resto è rumore.

5 • La votazione. È il passaggio che salva tutto. Il programma prende a caso due abbinamenti, calcola quale spostamento-rotazione-ingrandimento implicherebbero, e poi **conta quanti altri abbinamenti sarebbero d'accordo** con quella stessa trasformazione. Ripete l'operazione migliaia di volte e tiene la trasformazione che ha raccolto più consensi.

Gli abbinamenti d'accordo si chiamano *inlier*. Nel pannello 5 ne sono rimasti 52 su 856, e questa volta **sono tutti paralleli**: descrivono tutti lo stesso movimento. Questa procedura si chiama RANSAC, ed è ciò che permette di trovare la risposta giusta quando il 94% dei dati è sbagliato.

6 • Il risultato. La trasformazione trovata viene applicata all'immagine storica, che così si sovrappone al vettoriale. Nel pannello 6 i due strati vanno su colori complementari: **nero dove coincidono**, rosso dove c'è solo lo storico deformato, ciano dove c'è solo il vettoriale. Una registrazione corretta annerisce; un disallineamento si stacca in una frangia rosso-ciano, visibile anche per uno scarto di un paio di pixel.

2.3 Come facciamo a sapere se ha funzionato

Qui sta la particolarità di questo progetto. Entrambi i file portano con sé la propria **georeferenziazione**: sei numeri (il *world file*) che dicono a quali coordinate sul terreno corrisponde il pixel in alto a sinistra, e quanti metri misura il lato di un pixel. Da questi sei numeri si ricava per via puramente algebrica la trasformazione **esatta**, senza doverne indovinare nemmeno un pezzo.

Quindi la risposta giusta la conosciamo già, e possiamo dire di **quanti metri** il programma ha sbagliato. Nell'esempio della figura: 0.40 m — sotto l'incertezza del riferimento stesso, che di suo vale circa mezzo metro. Più in là di così non si può andare: la registrazione è buona quanto questa ground truth consente di misurare.

Il programma che stima non vede quei sei numeri. Li vede solo il codice che corregge. È una separazione imposta per costruzione (§4.2): se l'informazione sulla posizione filtrasse nell'algoritmo, il risultato non significherebbe più nulla, come uno studente che risolve il problema avendo visto le soluzioni.

2.4 Perché è difficile

Se la procedura sembra ragionevole, resta da capire perché non funzioni sempre. Tre ragioni, tutte misurate nei capitoli §8-§10:

- 1. I due disegni sono fatti di linee sottili, non di texture.** I metodi classici per riconoscere punti nascono per le fotografie, dove ogni zona ha un aspetto diverso dalle altre. Su un reticolo di confini, un incrocio somiglia a tutti gli altri incroci.
- 2. I due documenti non rappresentano la stessa realtà.** Il CXF è la mappa di oggi, il foglio è quella di un secolo fa: alcuni confini sono cambiati, e ci sono linee vettoriali che nell'inchiostro non esistono (§12.1).
- 3. L'aspetto è completamente diverso:** inchiostro sbiadito irregolare contro linee nette uniformi. È quello che in letteratura si chiama *divario di dominio*.

2.5 Glossario

termine	significato
registrazione	sovrapporre due immagini della stessa zona, trovando la trasformazione che porta l'una sull'altra
raster / immagine	griglia di pixel

termine	significato
vettoriale	descrizione per coordinate (elenchi di vertici), non per pixel
rasterizzare	disegnare un vettoriale su un'immagine
world file (.jgw)	sei numeri che legano i pixel di un'immagine alle coordinate sul terreno
georeferenziazione	il fatto che un'immagine sappia dove si trova sul terreno
ground truth	la risposta giusta, nota indipendentemente dall'algoritmo, contro cui si misura l'errore. Qui si ricava dai world file (§4)
H	la trasformazione: H_{true} è quella vera, H_{est} è quella stimata dal programma. Confrontarle è tutta la valutazione
warp	applicare una trasformazione a un'immagine, cioè ridisegnarla deformata secondo H
luminosità	il valore di grigio di un pixel: un solo numero, 0 (nero) – 255 (bianco)
gradiente	quanto e in che direzione la luminosità cambia in UN pixel, confrontandolo coi vicini (§7.1)
keypoint	un punto dell'immagine giudicato "riconoscibile"
descrittore	l'impronta numerica che riassume l'aspetto attorno a un keypoint, costruita raccogliendo i gradienti di molti pixel vicini
SIFT, ORB	due modi diversi di calcolare keypoint e descrittori
matching	accoppiare i keypoint di un'immagine con quelli dell'altra
ratio test di Lowe	il filtro di SIFT: tiene un abbinamento solo se il candidato migliore batte nettamente il secondo (§7.1)
RANSAC	la procedura a votazione del passo 5, che trova la risposta giusta anche con pochi dati buoni (dettaglio matematico in §7.2)
inlier	un abbinamento che concorda con la trasformazione scelta
outlier	un abbinamento che non concorda: su questi dati sono la grande maggioranza
inlier ratio	la percentuale di inlier: quanto erano buoni i dati di partenza
campione minimo	il numero di corrispondenze che bastano a calcolare esattamente una trasformazione (2, 3 o 4 a seconda della famiglia, §7.2)
omografia, affine, similarità	famiglie di trasformazioni con 8, 6 e 4 parametri liberi (§7.2)
binarizzazione	decidere, pixel per pixel, se è inchiostro o carta
Otsu, Sauvola	due modi di scegliere quella soglia (§6.1)
CLAHE	tecnica che non binarizza, ma aumenta il contrasto locale (§6.1)
LoFTR	rete neurale che abbina due immagini senza cercare prima i keypoint (§7.3)
RMSE	l'errore medio commesso, qui espresso in metri
checkpoint	i 100 punti su cui l'errore viene misurato, disposti a griglia regolare sull'immagine storica (§4)
pavimento	la soglia sotto la quale l'errore misurato non dice più nulla dell'algoritmo, perché è più piccolo dell'incertezza del riferimento: qui ~0.5 m (§4.1)
E1, E2, E3	i tre esperimenti: sintetico, reale, comparativo (§8, §9, §10)
degradazione	il guasto artificiale applicato alle immagini di E1 per misurare fin dove i metodi reggono; la scala è spiegata in §8
divario di dominio	il fatto che le due immagini, pur raffigurando la stessa zona, abbiano un aspetto completamente diverso
ablazione	togliere una componente e rimisurare, per vedere quanto contava

3. I dati

3.1 Il foglio e il vettoriale

Dal servizio "Consultazione dei fogli di mappa catastale" dell'Agenzia delle Entrate si ottengono, per il foglio 49 di Varazze:

file	contenuto
L675_004900.jpg	scansione dell'Originale di Impianto, 8489 × 5648 px
L675_004900.jgw	world file: la georeferenziazione della scansione
L675_004900.txt	metadati del ricampionamento
L675_004900.cxf	cartografia vigente in formato vettoriale, stesso sistema

Raster e vettoriale sono entrambi in **Cassini-Soldner zona G0007, origine Forte**

Diamante. Non è un EPSG (sistema di coordinate geografiche) standard e non serve riproiettare nulla: le coordinate dei due file sono già confrontabili.

E' necessario chiarire subito la differenza fra `.cxf` e `.jgw`, perché il nome fa pensare a due varianti dello stesso tipo di file, e non lo sono. Il **CXF** è un contenuto: un elenco di coordinate che disegnano i confini delle particelle, cioè la mappa moderna stessa scritta come numeri invece che come disegno.

Il **JGW non è** un contenuto, ma un'istruzione di conversione:

solli sei numeri, che dicono a quale coordinata reale sul terreno corrisponde il pixel in alto a sinistra della scansione e quanti metri misura il lato di un pixel — non contiene nessun confine, nessuna particella. Senza il JGW la scansione storica sarebbe un'immagine priva di qualunque legame con il territorio; senza il CXF non ci sarebbe una mappa moderna con cui confrontarla. Servono entrambi, e per ragioni complementari: il primo dà il contenuto da confrontare, il secondo dà la posizione sul terreno di ciò che si vede nella scansione — quest'ultima è anche l'ingrediente della ground truth (§4).

3.2 La trappola dei due CXF

Il servizio rilascia **due** file vettoriali per lo stesso foglio, e il file `_SistemaDiRappresentazione.txt` li distingue così:

```
L675_004900 (nativamente CASSINI-SOLDNER zona G0007 - Forte Diamante - 02)
L675_00490Z (nativamente ROMA40-GAUSS BOAGA DA RILIEVO AEROFOTOGRAMMETRICO OVEST | EPSG:3003)
```

Solo **L675_004900** condivide il sistema di riferimento del JGW. Caricare **L675_00490Z** insieme al raster produce due strati a chilometri di distanza e una diagnosi che parte dal posto sbagliato. È un errore che costa un pomeriggio e che nessun messaggio d'errore segnala: i file si aprono entrambi senza problemi, semplicemente descrivono posti diversi.

3.3 Il parser CXF e il campo che sfasa tutto

Il CXF è un formato testuale elementare: un campo per riga, CRLF, codifica latin-1. Il parser è scritto a mano in una trentina di righe, senza librerie. Ogni poligono è un record **BORDO** che dichiara un nome — il numero di particella, oppure **ACQUA**, **STRADA**, il nome del foglio — e un **codice** che ne dice la natura: **18** particella, **12** acqua, strada o bordo del foglio, **25** un unico caso residuo. Sono i codici che compaiono nelle tabelle di §9 come scelta di che cosa disegnare nel raster moderno.

Poi, fra l'intestazione e l'elenco dei vertici, si intromette un campo di servizio: `nf1ag`, che dichiara quanti indici extra seguono prima che le coordinate comincino davvero.

Ed è la trappola. Vale 0 in 838 record su 871, ma 1, 2 o 5 nei restanti 33. Chi lo ignora e comincia a leggere le coordinate subito dopo il conteggio dei vertici ottiene un parser che funziona benissimo per il 96% dei record e sfasa sul restante 4%: il sintomo è che l'estensione del foglio passa da ~1.6 km a ~31 km, con coordinate positive dove dovrebbero essere tutte negative.

Il progetto ha due difese contro questo errore, entrambe automatiche ed entrambe eseguite a ogni parsing, non solo nei test:

- 1. **Assert di estensione:** dopo il parsing, ogni singolo vertice deve cadere dentro l'estensione del raster calcolata dal JGW. È il controllo che smaschera lo sfasamento da `nflag`: le coordinate lette al posto sbagliato sono piccoli indici (0-50), non numeri dell'ordine di -30000, e sfiorano l'estensione alla prima occorrenza.
- 2. **Sentinella di segno:** nessuna coordinata può essere positiva, perché il foglio sta interamente nel terzo quadrante del sistema.

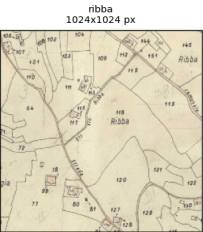
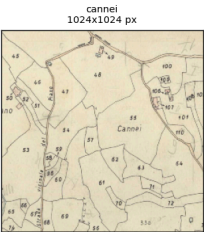
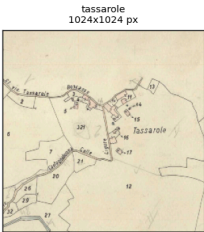
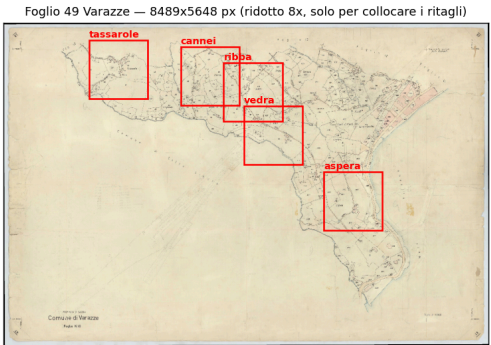
3.4 I ritagli

Il foglio intero è troppo grande per essere utile allo scopo: tiling e gestione della memoria non aggiungono nulla al tema. Si lavora su cinque ritagli di circa 1000-1200 px per lato, scelti nella fascia collinare dove il vettoriale copre per intero il contenuto disegnato.

Due zone sono state **escluse deliberatamente**:

- il **territorio di Celle Ligure**, fascia bianca a ovest: fuori giurisdizione, vuoto in entrambe le rappresentazioni;
- l'**ente urbano** sulla costa: nel catasto terreni le zone urbane sono rimandate al catasto urbano, quindi il vettoriale non le copre, mentre l'impianto ha lì il tratto più fitto. È il divario di dominio massimo, ma per la ragione sbagliata: non misurerebbe la capacità del metodo, solo l'assenza di un riferimento.

crop	x0	y0	larghezza	altezza	X (m)	Y (m)
tassarole	1500	300	1024	1024	-31098 ... -30838	-11616 ... -11355
cannei	3100	420	1024	1024	-30691 ... -30431	-11646 ... -11386
ribba	3850	700	1024	1024	-30500 ... -30240	-11717 ... -11457
vedra	4200	1450	1024	1024	-30411 ... -30151	-11908 ... -11648
aspera	5600	2600	1024	1024	-30055 ... -29795	-12201 ... -11940



Ogni ritaglio è salvato come PNG accompagnato dal **proprio world file**, ottenuto da quello del foglio spostandone l'origine sul primo pixel del ritaglio. Ogni ritaglio sa quindi dove si trova sul terreno esattamente come il foglio intero, ed è ciò che rende possibile la ground truth del capitolo seguente.

4. La ground truth: esatta e gratuita

Questo è il punto metodologico su cui poggia tutto il resto.

Il world file `L675_004900.jgw` contiene sei numeri che definiscono una trasformazione affine da pixel a coordinate: la dimensione del pixel sui due assi — **0.254453 m**, cioè la scala 1:2000 — due termini di rotazione, qui nulli, e le coordinate del centro del primo pixel, (-31480.044315, -11278.758056). Da questi sei numeri la posizione sul terreno di qualunque pixel si ottiene con due moltiplicazioni e due somme.

Rasterizzando il CXF su una griglia di cui conosciamo, allo stesso modo, il legame fra pixel e coordinate, si ottengono due trasformazioni note, e la trasformazione di riferimento è la loro composizione:

$$H_{true} = \frac{W_{storico}}{W_{moderno}}$$

Non è stato annotato un solo punto di controllo a mano. La ground truth è analitica, esatta per costruzione, e `evaluate.py` la usa per produrre l'RMSE in metri su una griglia regolare di checkpoint nell'immagine storica (10×10 punti, bordi esclusi):

$$e_i = \|H_{est} \cdot p_i - H_{true} \cdot p_i\|_2$$
$$RMSE_m = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2} \cdot r_{dest}$$

dove $N = 100$ sono i checkpoint e r_{dest} è la risoluzione in metri per pixel della griglia di arrivo: 0.254453 m/px in E1, dove la coppia è il ritaglio contro se stesso trasformato, e 0.20 m/px in E2, dove l'arrivo è il raster del vettoriale (§9.1).

cioè: si applicano entrambe le trasformazioni — quella stimata e quella vera — agli stessi 100 punti, si misura di quanto le due risposte divergono, e si converte in metri. È l'unico numero su cui si giudica l'intera pipeline: ogni tabella dei capitoli successivi è, in ultima analisi, un'aggregazione di questa formula su configurazioni diverse.

La correttezza della composizione di `H_true` è verificata da un test: un punto trasformato avanti e indietro torna su sé stesso entro $1.1 \cdot 10^{-13} px$, contro la soglia dichiarata di $10^{-9} px$.

4.1 L'incertezza del riferimento, dichiarata

Il JGW non è verità assoluta. I metadati del foglio dicono che la scansione è stata ricampionata su **76 coppie omologhe**, con:

grandezza	valore
scarto massimo	1.28 m
scarto medio	0.56 m
deviazione standard	0.25 m

Questo definisce un **pavimento**: un RMSE inferiore a ~0.5 m non misura più l'errore dell'algoritmo, misura il rumore del riferimento. Va tenuto presente ogni volta che si legge una cifra nelle tabelle di §9.

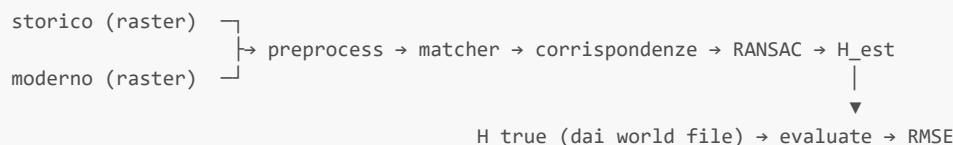
4.2 Separazione fisica fra algoritmo e riferimento

Un rischio serio in un lavoro di questo tipo è che l'informazione di georeferenziazione filtri nella pipeline che dovrebbe stimarla — a quel punto il risultato non significherebbe più nulla. Il progetto lo impedisce per costruzione:

- `pipeline.py` riceve due array di pixel e non importa né `io_geo` né `groundtruth`;

- solo `evaluate.py` vede entrambi i mondi;
- un **test automatico** ispeziona il codice dei moduli dell'algoritmo e fallisce se uno di essi importa la georeferenziazione;
- la CLI, invocata **senza** i parametri `--jgw=*`, produce comunque `H_est` e semplicemente non calcola l'RMSE. È la prova architetturale, eseguibile in qualsiasi momento.

5. La pipeline



Il preprocessing si applica **a entrambe le immagini allo stesso modo**: fa parte della pipeline, non è un ritocco di una delle due. Il matcher è scelto per nome da una fabbrica, e la pipeline non sa quale stia usando: è ciò che rende confrontabili il classico e il neurale cambiando un solo parametro.

Sotto le 4 corrispondenze la stima fallisce in modo pulito — `success=False` con il motivo — e la riga di CSV viene scritta lo stesso. Un esperimento che non produce una stima ha comunque prodotto un dato.

Ogni esecuzione è **deterministica**: il seed di RANSAC è fissato con `cv2.setRNGSeed`, ogni generazione sintetica passa da un generatore seminato. Eseguendo due volte la stessa griglia si ottengono CSV identici cifra per cifra su ogni colonna tranne i tempi.

5.1 Come si leggono le tabelle e i grafici

Tutte le tabelle dei capitoli §8-§10 sono aggregazioni delle stesse righe di CSV e usano gli stessi nomi di colonna. Conviene fissarne il significato una volta sola.

colonna	significato
<code>prove</code>	quante esecuzioni la cella riassume. Una prova è una registrazione completa: un ritaglio, una trasformazione, una configurazione
<code>successo_pct</code>	percentuale di prove in cui la stima è riuscita e l'errore è rimasto sotto la soglia dichiarata: 0.2545 m su E1 (dove la ground truth è esatta e si può essere severi), 2 m su E2
<code>rmse..._mediano</code>	errore mediano su tutte le prove, fallimenti compresi. È grande di proposito: su E2 una configurazione può non riuscire mai, e allora "l'errore sulle riuscite" non esisterebbe
<code>..._ok</code>	la stessa grandezza calcolata sulle sole prove riuscite (<code>rmse_px_mediano_ok</code> , <code>rmse_m_mediano_ok</code>). Le due cifre per la stessa configurazione differiscono: non è un'incoerenza, sono due domande diverse
<code>rmse_m_minimo</code>	il caso migliore della cella: dove arriva il metodo quando le cose vanno bene
<code>inlier_ratio</code>	la frazione di corrispondenze giudicate coerenti con la trasformazione scelta — quanto erano buoni i dati che RANSAC ha ricevuto
<code>match_medi,</code> <code>match_mediani, n_matches</code>	quante corrispondenze il matcher ha prodotto, prima di RANSAC
<code>t_ms</code>	millisecondi del solo matching . Non comprende il preprocessing, né RANSAC, né il caricamento del modello neurale: il CSV li cronometra in colonne separate, e tenerli distinti è ciò che rende leggibile il confronto di §10.1
<code>preprocess: none</code>	nessuna elaborazione: l'immagine in toni di grigio così com'è
<code>codici</code>	quali oggetti del CXF sono stati disegnati nel raster moderno: 18 = le sole particelle, 18+12 = particelle più acque e strade (§3.3)
<code>esperimento, config</code>	quale esperimento ha prodotto la riga, e in forma compatta la coppia preprocessing / modello geometrico

colonna	significato
<code>rmse_m, success</code>	nella sola tabella di §9.5 non c'è aggregazione: ogni riga è una registrazione, quindi l'errore e l'esito sono quelli di quella prova

Le tabelle riportano sempre la **mediana** e il tasso di successo, mai la media: la ragione è in §8.3, e riguarda il fatto che una stima sbagliata di migliaia di pixel trascinerebbe da sola la media di tutto il gruppo.

I **grafici** seguono la stessa convenzione. La linea è la mediana; la banda attorno è l'intervallo fra il primo e il terzo quartile, cioè la fascia in cui cade la metà centrale delle prove — una misura di dispersione che i valori estremi non gonfiano. L'asse dell'errore è in **scala logaritmica**, perché sullo stesso grafico convivono errori di centesimi di pixel e di migliaia: in scala lineare i primi sarebbero indistinguibili da zero.

6. Preprocessing

Il preprocessing è il cuore della componente classica: è il passo 2 del capitolo §2, quello che decide per ogni pixel se è inchiostro o carta. Ogni stadio è attivabile da riga di comando, perché confrontare le alternative è uno dei risultati, non un parametro da fissare una volta per tutte.

6.1 Le tre strade, spiegate

Il problema. Su una fotografia normale si potrebbe scegliere un valore di grigio e dire: "più scuro di così è inchiostro, più chiaro è carta". Su un foglio d'archivio non funziona, perché la carta non ha un colore solo: è più gialla nelle zone ingiallite, più scura vicino alle pieghe, più chiara al centro. Una soglia che va bene in un angolo del foglio sbaglia nell'angolo opposto.

Una precisazione sulle cifre che seguono. Binarizzare non toglie né aggiunge pixel: l'immagine ne ha esattamente gli stessi prima e dopo. Cambia l'*etichetta* — ogni pixel passa da un valore fra 0 e 255 all'appartenenza a una di due classi, inchiostro o carta. È una classificazione, non una cancellazione, e per questo la percentuale di pixel dichiarati inchiostro può salire o scendere senza che ciò sia di per sé un miglioramento. Quando sale di molto è di norma un guasto: Otsu sotto gradiente di illuminazione arriva al 43.66% (§6.3), e quel 43.66% è la carta scurita promossa a inchiostro.

Otsu — una soglia sola per tutta l'immagine. Guarda l'istogramma dei grigi e sceglie il valore che separa meglio le due popolazioni (scuri e chiari). È il metodo classico di riferimento, e serve qui come termine di paragone. Il suo limite è strutturale: se la carta cambia luminosità da una parte all'altra, una soglia sola non può andare bene ovunque.

Sauvola — una soglia diversa per ogni zona. Per ogni pixel guarda solo un quadratino di 25×25 pixel attorno a sé e calcola lì la sua soglia, con la formula

$$T = m \cdot [1 + k \cdot (s/R - 1)]$$

con m = media locale, s = deviazione locale

Dove la carta è uniforme, s è piccola e la soglia si abbassa, così le irregolarità di fondo non vengono scambiate per tratto. Dove c'è disegno, s cresce e la soglia si alza verso la media. In pratica: **si adatta alla carta, zona per zona.**

Fatta ingenuamente costerebbe un ricalcolo per ogni pixel su tutta la finestra; qui è implementata con le *immagini integrali*, una tabella precalcolata che permette di ottenere media e varianza di qualunque riquadro con poche operazioni. Il risultato è identico, ma il costo non dipende più dalla dimensione della finestra. La correttezza dell'ottimizzazione è verificata confrontandola pixel per pixel con la versione ingenua — è il genere di errore che non si vedrebbe mai guardando l'immagine.

CLAHE — non binarizzare affatto. È l'alternativa: invece di ridurre tutto a bianco e nero, aumenta il contrasto **localmente**, così i tratti sbiaditi diventano più leggibili senza buttare via le sfumature. La ragione per provarlo è concreta: SIFT lavora sulle sfumature di grigio, e binarizzare potrebbe distruggere proprio l'informazione su cui si basa. Quale delle due strade convenga non si può decidere a tavolino, ed è una delle domande sperimentali del progetto (§6.2 e §9).

Le domande sperimentali sono due, e sono indipendenti. Otsu contro Sauvola confronta *soglia globale contro soglia locale*. {Otsu, Sauvola} contro CLAHE confronta invece *binarizzare contro non binarizzare*, che è un asse a sé: anche CLAHE lavora zona per zona, e la ragione per provarlo non è l'illuminazione ma la perdita delle sfumature su cui SIFT costruisce il descrittore.

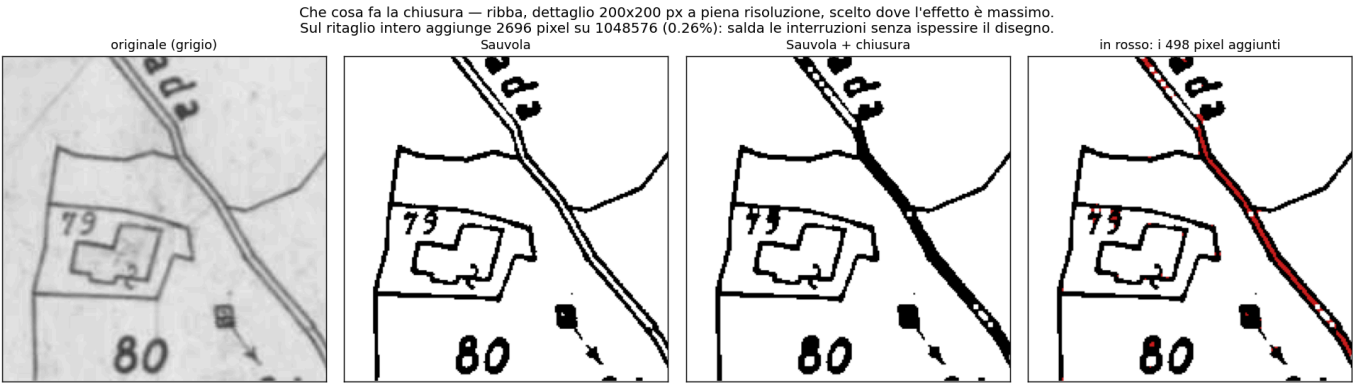
Morfologia — ritoccare la forma del tratto. Due operazioni elementari: l'*apertura* cancella i puntini isolati (la grana della carta scambiata per inchiostro), la *chiusura* ricongiunge i tratti interrotti, frequenti perché il pennino stacca. L'ordine conta: si apre prima e si chiude poi, perché chiudendo per primo si salderebbe la grana al tratto, rendendola poi non più rimovibile.

Non è una precauzione teorica, e si vede in un numero. Sul ritaglio **ribba**, partendo da Sauvola:

variante	pixel etichettati inchiostro
Sauvola	6.84%
Sauvola + chiusura	7.10%
Sauvola + chiusura + rimozione delle componenti piccole	7.09%

La chiusura aggiunge inchiostro, come deve. Ma il passo successivo — quello che elimina le componenti connesse più piccole di una soglia, cioè proprio la grana — ne recupera **0.01 punti**, un centinaio di pixel su un milione: praticamente nulla. Il motivo è che quella variante ha la chiusura e **non** l'apertura, quindi la grana non viene mai tolta e la chiusura la salda al tratto: smette di essere una componente piccola e isolata e diventa parte di una componente grande, che il filtro sull'area non tocca più. Aprire per primo non è una questione di stile, è ciò che rende possibile la pulizia a valle.

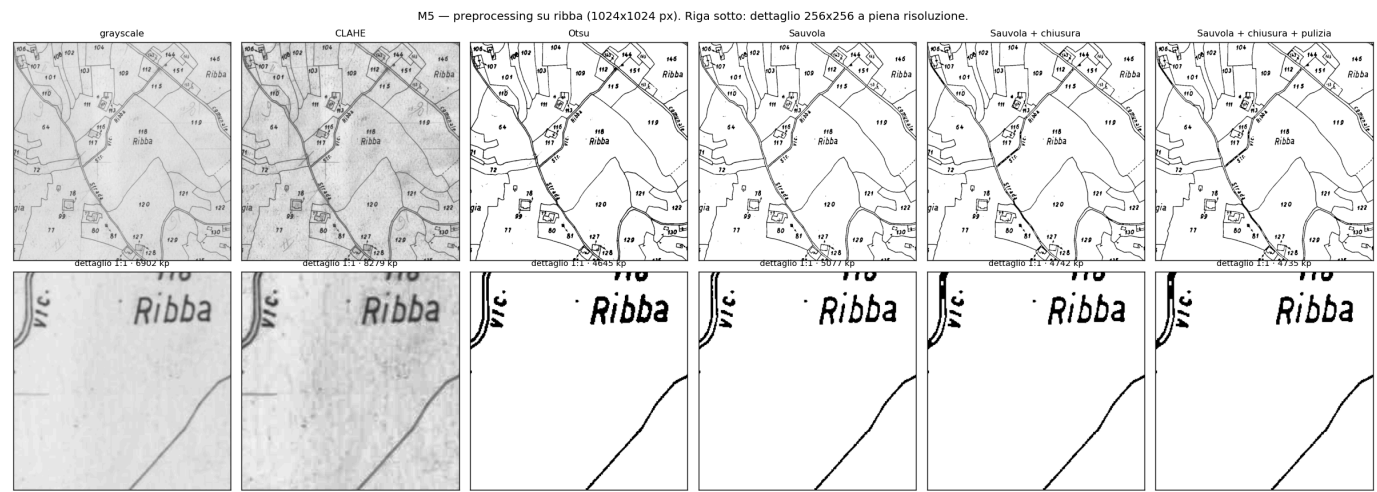
La variante che nelle tabelle compare come **sauvola+chiusura** è esattamente questo: la stessa binarizzazione, più un passaggio di chiusura con un elemento strutturante di 3×3 pixel. In pratica il tratto viene dilatato di un pixel e poi ristretto: le interruzioni più strette di quel pixel restano riempite, tutto il resto torna com'era.



Sull'intero ritaglio la chiusura aggiunge 2696 pixel su 1 048 576, lo 0.26%: è un ritocco, non una trasformazione. Ma il quarto pannello mostra anche un effetto collaterale che non era previsto e che va detto: **dove due linee corrono parallele e vicine — il caso delle strade, disegnate a doppio bordo — la chiusura**

le fonde in una fascia piena. La strada smette di essere due tratti sottili e diventa un nastro nero.

Non è necessariamente un male: nel vettoriale le strade sono poligoni, quindi riempirle avvicina le due rappresentazioni. Ma cambia la natura di ciò che il matcher vede, ed è parte della ragione per cui questa variante si comporta diversamente dalle altre nei risultati di §9 e §10.



Le figure hanno una riga di dettagli **a piena risoluzione**, perché su una binarizzazione il ridimensionamento inganna sempre: un tratto interrotto e uno saldato diventano indistinguibili a un quarto di scala.

6.2 Quanto costa binarizzare

Contando i keypoint SIFT rilevati su ciascuna variante, rapportati al grayscale:

variante	keypoint (rapporto sul grayscale)
CLAHE	1.17× – 1.37×
Sauvola	0.74× – 0.79×
Sauvola + chiusura	0.69× – 0.77×
Otsu	0.67× – 0.72×

Il dato è consistente su tutti e cinque i ritagli: **binarizzare costa fra il 21 e il 33% dei keypoint, CLAHE ne aggiunge fra il 17 e il 37%**. Non è ancora la risposta — quella è l'RMSE di §8 e §9 — ma indica la direzione.

6.3 Una previsione verificata invece che assunta

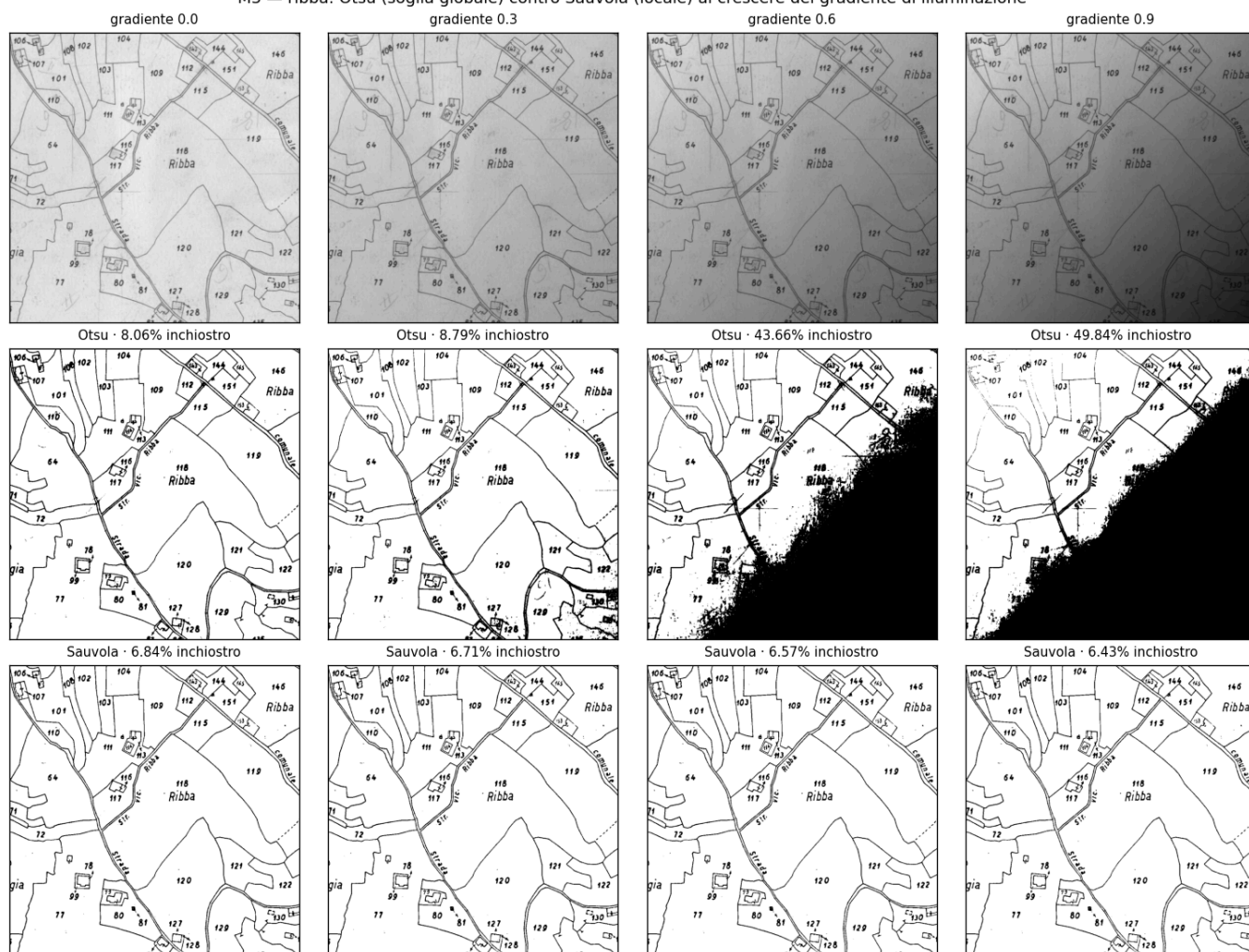
L'impostazione del progetto prevedeva che Otsu fallisse "su carta ingiallita con gradiente di illuminazione". **Sui ritagli reali questo non accade:** sono illuminati in modo uniforme e Otsu si comporta benissimo.

Invece di dare per buona la previsione o di dichiararla smentita, la si è messa alla prova in modo controllato, aggiungendo un gradiente di illuminazione sintetico di intensità crescente:

gradiente	Otsu	Sauvola
0.0	8.06%	6.84%
0.3	8.79%	6.71%
0.6	43.66%	6.57%
0.9	49.84%	6.43%

(percentuale di pixel classificati come inchiostro, ritaglio **ribba**)

M5 — ribba: Otsu (soglia globale) contro Sauvola (locale) al crescere del gradiente di illuminazione



A gradiente 0.6 la soglia globale collassa — classifica come inchiostro metà immagine, cioè la carta scurita — mentre Sauvola resta piatta lungo tutta la scala. **La previsione è corretta in generale, ma il suo antecedente su questi dati non si verifica.** Sauvola è più robusta, e lo si dimostra; su questo foglio specifico quella robustezza non serve, e anche questo è un dato.

7. Matching e stima

Sono i passi 4 e 5 del capitolo §2. L'interfaccia è la stessa per tutti i metodi: due immagini entrano, due insiemi di punti corrispondenti escono.

7.1 Trovare gli abbinamenti

Prima un chiarimento, perché qui si confondono facilmente tre livelli diversi. Un pixel ha una **luminosità**: un solo numero, da 0 (nero) a 255 (bianco). Il **gradiente** è un'altra cosa, ed è calcolato *per ogni singolo pixel* confrontandolo con i vicini immediati: è un vettore che dice di quanto e in quale direzione la luminosità cambia proprio lì. Dove la carta è uniforme il gradiente è quasi nullo; dove un pixel sta sul bordo di un tratto d'inchiostro, il gradiente è grande e punta perpendicolare al bordo.

Da un pixel al descrittore — ribba, dettaglio 140x140 px a piena risoluzione.
 Un numero per pixel (1) → un vettore per pixel (2+3) → un istogramma per l'intero riquadro (4).
 Il vero descrittore SIFT ripete il pannello 4 su 16 sotto-finestre e le incolla: $16 \times 8 = 128$ numeri.

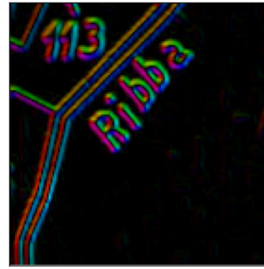
1 - luminosità
 (un numero per pixel: 0=nero, 255=bianco)



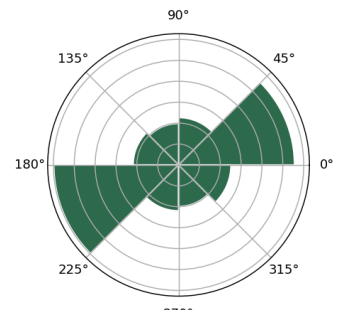
2 - magnitudine del gradiente
 (quanto è forte il bordo, lì)



3 - direzione del gradiente
 (colore = verso; nero = carta uniforme)



4 - l'istogramma di orientazioni
 (8 direzioni, pesate per magnitudine)



Il pannello 2 mostra quanto è forte il gradiente in ogni punto di un dettaglio reale: è acceso solo lungo i contorni delle lettere, nero altrove — la carta uniforme non genera gradiente. Il pannello 3 mostra la sua direzione, colorata: lungo un bordo curvo il colore ruota con esso. Il pannello 4 raccoglie i gradienti di *tutta* la finestra in un istogramma a 8 direzioni, pesato da quanto è forte ciascuno: è la stessa aggregazione che sta dentro il descrittore SIFT.

Con che cosa è calcolato quel gradiente. La figura usa l'operatore di **Sobel**, due kernel 3×3 che stimano la derivata orizzontale e verticale. SIFT internamente non usa Sobel ma la semplice differenza fra il pixel precedente e il successivo, e la ragione è istruttiva: il kernel di Sobel si fattorizza in $\begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$, cioè è quella differenza più una lisciatura nella direzione ortogonale. SIFT quella lisciatura ce l'ha già, perché lavora sull'immagine convoluta con una gaussiana (qui sotto), e applicare Sobel significherebbe lisciare due volte. Nella figura serve invece, perché il dettaglio è preso dalla scansione grezza, con la grana della carta.

Come SIFT trova i punti, prima di descriverli. Sfoca la stessa immagine con gaussiane di ampiezza crescente e ne **sottrae** le versioni consecutive: questa *differenza di gaussiane* è un'approssimazione del **laplaciano** della gaussiana, e i suoi massimi sono i punti cercati. Da qui viene tutto il resto — l'invarianza di scala, perché il punto si cerca su tutte le sfocature insieme e non su una sola; e il fatto, decisivo per questo progetto, che il laplaciano risponda ai **blob**, cioè a macchie con un massimo di curvatura, e non ai bordi. È la ragione strutturale per cui un reticolo di linee sottili è il terreno peggiore per SIFT, che §7.2 e §9 constatano poi nei numeri.

A ogni punto SIFT assegna poi un **orientamento dominante**, ricavato dagli stessi gradienti, e ruota di conseguenza tutto ciò che legge intorno. È da lì che viene l'invarianza alla rotazione — costruita per progetto, non appresa dai dati, e §8.1 mostra che è proprio questa la differenza che LoFTR non ha.

Il descrittore, infine. Per ogni punto trovato SIFT procede così: prende una finestra di 16×16 pixel attorno al punto, la divide in 16 sotto-finestre di 4×4 , e per ciascuna calcola l'istogramma a 8 direzioni del pannello 4. Sedici istogrammi da 8 numeri, incollati uno dopo l'altro, fanno **128 numeri**: è il descrittore, un solo vettore per l'intero punto, non uno per pixel. Due punti che nelle due immagini hanno descrittori simili sono candidati alla stessa cosa reale.

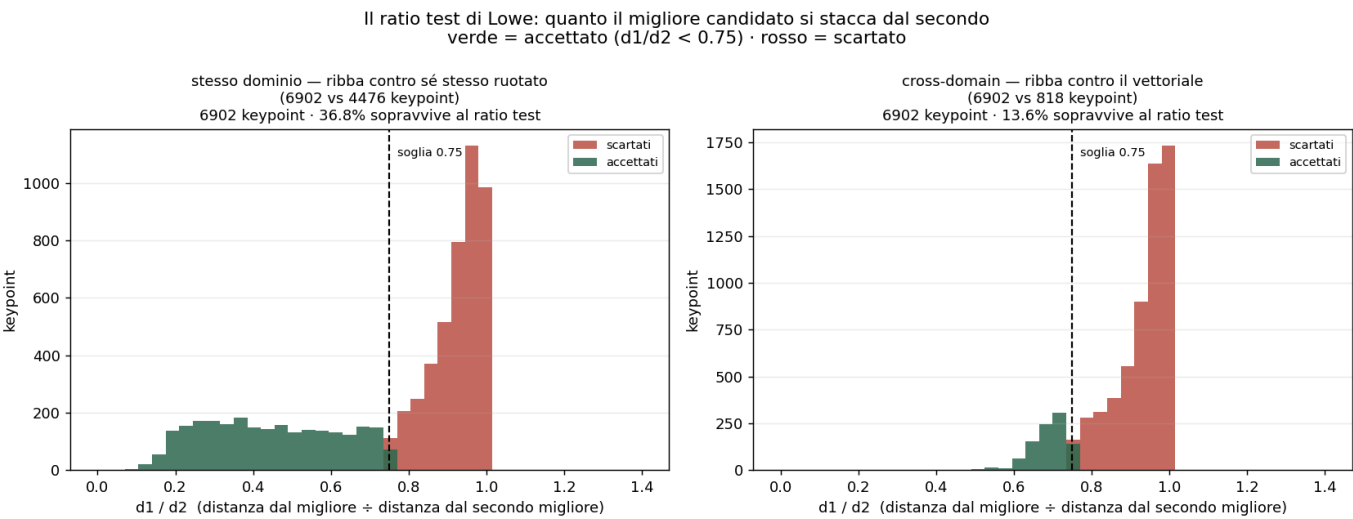
Il problema è che un candidato "abbastanza simile" spesso non basta. Per questo si usa il **ratio test di Lowe**: per ogni punto si guardano i *due* candidati migliori nell'altra immagine, **d1** (distanza dal più simile) e **d2** (distanza dal secondo più simile), e l'abbinamento si accetta solo se

$$d1 < 0.75 \cdot d2$$

cioè se il migliore è nettamente più vicino del secondo, non di poco. L'idea è che se i due candidati migliori si somigliano fra loro (**d1** e **d2** vicini),

allora quel punto non è davvero distintivo — ci sono almeno due posti dove potrebbe stare — ed è meglio scartarlo che rischiare un abbinamento a caso.

Due esempi con numeri: se $d1 = 50$ e $d2 = 51$, il rapporto è 0.98, molto sopra la soglia — **scartato**, i due candidati sono quasi indistinguibili. Se $d1 = 50$ e $d2 = 100$, il rapporto è 0.50 — **accettato**, il migliore si stacca nettamente.



La figura mostra questo rapporto calcolato su ogni keypoint di un ritaglio reale, in due situazioni. **Contro sé stesso ruotato** (stesso dominio, come in E1) la distribuzione è ampia e il 36.8% dei punti supera la soglia: ci sono molti candidati chiaramente migliori degli altri. **Contro il vettoriale** (cross-domain, come in E2) la distribuzione si sposta visibilmente sopra 0.75, e **solo il 13.6% dei keypoint sopravvive** — poco più di un terzo della quota dello stesso dominio. È coerente con le poche centinaia di corrispondenze che SIFT trova su E2 (§9.3): non è che il ratio test sia mal tarato, è che nel cross-domain molto più raramente un punto ha davvero un solo candidato nettamente migliore di tutti gli altri — sono più vagamente simili a molti altri punti che nello stesso dominio.

ORB fa la stessa cosa in modo più rapido ed essenziale: descrive ogni punto con una stringa di bit invece che con 128 numeri, e confronta le stringhe contando i bit diversi. Al posto del ratio test usa il **cross-check**: tiene l'abbinamento solo se A sceglie B e B sceglie A.

Questi due filtri hanno una conseguenza importante, che emergerà in §9.3: il ratio test è molto più selettivo del cross-check, e su dati difficili lascia molte meno corrispondenze.

7.2 Scegliere quanto la trasformazione può deformare

RANSAC — la votazione descritta al passo 5 — non cerca "una trasformazione qualsiasi": cerca la migliore all'interno di una **famiglia** che decidiamo noi. Ne sono previste tre, di libertà crescente:

famiglia	parametri	cosa può fare
similarità	4	spostare, ruotare, ingrandire — le forme restano simili a sé stesse
affine	6	in più: stirare in una direzione, inclinare (un quadrato diventa un parallelogramma)
omografia	8	in più: la deformazione prospettica di un piano visto di sbieco

Più parametri significa poter descrivere trasformazioni più complicate. Ma significa anche, e questo è il punto, **più modi di mettersi d'accordo con dati sbagliati**: con più libertà RANSAC può trovare un consenso anche fra abbinamenti casuali. Il confronto fra le tre famiglie non è un dettaglio

implementativo: come si vedrà in §9.4, su questi dati è il fattore che decide fra successo e fallimento.

Sotto le 4 corrispondenze la stima si ferma dichiarando il fallimento, invece di restituire un risultato che non avrebbe senso.

Come funziona il voto, con i numeri

Un'iterazione di RANSAC fa quattro cose. **Pesca** a caso un *campione minimo*: il numero s di corrispondenze che bastano a calcolare la trasformazione esattamente — 2 per la similarità, 3 per l'affine, 4 per l'omografia, cioè metà dei parametri, perché ogni punto ne fissa due. **Risolve** il sistema esatto, ottenendo una H candidata. **Applica** quella H a tutte le corrispondenze e conta quante cadono vicino al punto con cui erano state abbinate. **Ripete**, tenendo alla fine la H che ha raccolto più consensi.

Il test su una singola corrispondenza è elementare, e vale la pena scriverlo perché è facile confonderlo con la formula qui sotto: si applica la H candidata al punto di partenza, si misura quanto lontano cade dal suo compagno, e la coppia si tiene se quella distanza sta sotto una soglia — **3 pixel**, il valore di `--ransac-thresh`.

$$\|H \cdot a - b\| < 3 \text{ px}$$

dove la coppia è un inlier

La famiglia entra nei primi due passi, non nel terzo. Decide quante corrispondenze servono per costruire un candidato e che forme quel candidato può assumere; la soglia invece resta 3 px per tutte e tre. Non cambia quindi il *test*, cambia lo **spazio delle ipotesi sottoposte al test** — ed è qui che nasce il risultato di §9.4: una H sbagliata, pescata da una famiglia più libera, riesce a portare per caso più coppie sotto i 3 px, perché ha più parametri con cui contorcersi. La soglia è identica, è il candidato a essere più capace di ingannarla.

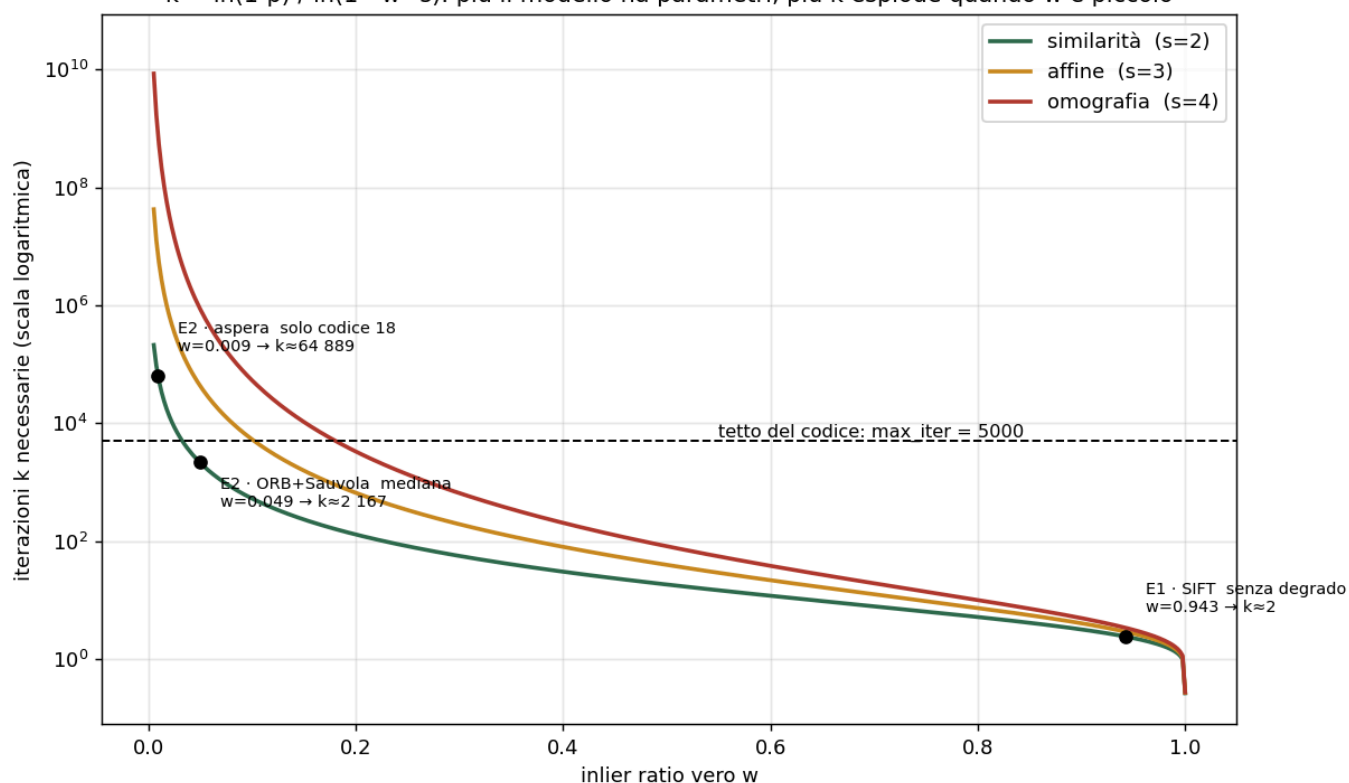
Un ultimo passo spiega la precisione che si ottiene. La H vincente **non resta quella calcolata dal campione minimo**: viene ricalcolata sull'insieme completo dei suoi inlier. Le 2-4 corrispondenze del campione servono a *trovare* il consenso, non a fornire la risposta — la stima definitiva si costruisce su tutte le centinaia di coppie che quel consenso ha raccolto, ed è la ragione per cui da un campione di quattro punti si arriva a un errore sub-pixel.

Resta la domanda su **quante** iterazioni servano perché prima o poi capiti un campione fatto di corrispondenze tutte corrette. Detta w la frazione di corrispondenze corrette e p la sicurezza voluta (qui 0.995):

$$k \geq \ln(1 - p) / \ln(1 - w^s)$$

Questa è un **budget di tentativi**, non un criterio di selezione: non giudica nessuna coppia. E non è valutabile in anticipo, perché w è precisamente ciò che non si sa — scoprirlo è il lavoro di RANSAC. Per questo il codice porta un tetto fisso, `max_iter = 5000`, e la formula serve a stabilire *a posteriori* se quel tetto era sufficiente. La sua lezione è una sola, ma decisiva: w è elevato a s , quindi il costo non cresce con il numero di parametri, **esplode**.

Iterazioni richieste da RANSAC per una confidenza del 99.5%
 $k = \ln(1-p) / \ln(1 - w^s)$: più il modello ha parametri, più k esplode quando w è piccolo



Il grafico riporta la curva per le tre famiglie, il tetto effettivo del codice (5000 iterazioni) e tre casi realmente misurati negli esperimenti. Il confronto è brutale:

- **E1, SIFT senza degrado** ($w = 0.960$): bastano **2** iterazioni. Quasi tutte le corrispondenze sono corrette, il primo campione è già pulito.
- **E2, ORB + Sauvola** ($w = 0.035$): la similarità ne chiede **4304**, appena dentro il budget; l'omografia, con gli stessi dati, ne chiederebbe **oltre tre milioni e mezzo**. È la ragione algebrica del risultato di §9.4: con pochi inlier un modello a più parametri non è "più difficile da stimare bene", è **irraggiungibile** nel numero di tentativi concesso.
- **E2, ritaglio aspera con le sole particelle** ($w = 0.009$, il caso fallito di §9.5): perfino la similarità ne chiederebbe **oltre 65 000**. Il codice ne prova 5000 e si ferma: non è che RANSAC abbia cercato e non abbia trovato, è che non ha mai avuto una possibilità realistica.

7.3 Il matcher neurale (componente B)

LoFTR affronta il problema da un'angolazione diversa. I metodi classici procedono in due tempi: prima *trovano* i punti notevoli, poi li descrivono. Se il primo passo fallisce — e su un disegno al tratto può fallire, perché non ci sono angoli e macchie ben definiti — il secondo non ha nulla su cui lavorare.

LoFTR salta il primo passo: è *detector-free*. Divide entrambe le immagini in una griglia regolare e confronta ogni cella di una con tutte le celle dell'altra, con un meccanismo di attenzione che tiene conto anche del contesto circostante. È una rete neurale già addestrata su fotografie: qui viene usata così com'è, senza alcun riaddestramento.

L'aspettativa che giustifica il confronto è precisa: **dovrebbe funzionare proprio dove i metodi classici soffrono**, cioè dove non ci sono punti notevoli da trovare. Se questa aspettativa si realizzi su mappe catastali è la domanda della componente comparativa, e la risposta è in §10.

8. E1 — esperimento sintetico, stesso dominio

Il primo esperimento confronta un ritaglio con **sé stesso trasformato con una H nota** — rotazione, scala, traslazione, lieve prospettiva — al quale si può aggiungere una **degradazione** graduale. La ground truth è esatta per costruzione e il divario di dominio è assente.

Serve a due cose, entrambe indispensabili: stabilire il **tetto di prestazione** dei matcher, e verificare che la pipeline sia **corretta**. Se E1 fallisce, il problema è nel codice.

Che cosa significa «degradazione 1.0»

Il numero che compare nelle tabelle e sull'asse dei grafici è una sola manopola che comanda quattro difetti insieme, ciascuno imitazione di un problema reale di una scansione d'archivio:

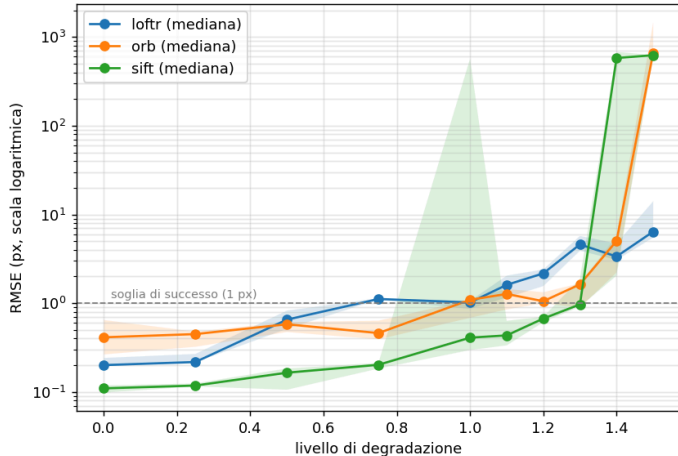
componente	che cosa imita	a livello 1.0
rumore	la grana della carta e del sensore	scarto di 20 livelli di grigio
sfocatura	la messa a fuoco imperfetta, la carta ondulata	sfocatura gaussiana di 2 pixel
contrasto	l'inchiostro sbiadito, la carta ingiallita	dimezzato
luminosità	l'illuminazione non uniforme della scansione	+20 livelli

Sono comandati da un numero solo perché la figura ha un asse solo: con quattro parametri liberi non ci sarebbe una curva, ci sarebbe una nuvola in quattro dimensioni. In concreto, a **1.0** l'inchiostro nero non è più nero e la carta bianca non è più bianca: la scala di grigi si dimezza, con sopra rumore e sfocatura. A **1.5** ne resta un quarto, e il rumore da solo ne copre metà — è lì che i matcher cedono (§8.2).

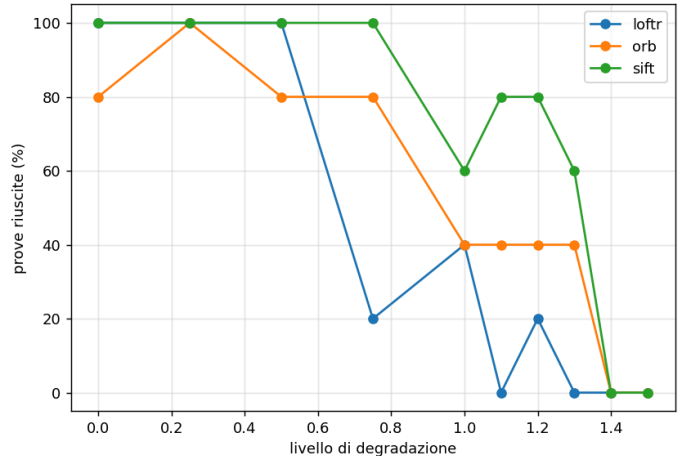
Due precisazioni: la degradazione agisce sui toni di grigio e non tocca la geometria, che resta un asse indipendente — e si applica a **una sola** delle due immagini. La coppia è quindi asimmetrica: riferimento pulito contro scansione rovinata, che è la stessa asimmetria di E2, ma qui con ground truth esatta e livello di degrado noto.

matcher	preprocess	prove	successo_pct	rmse_px_mediano_ok	rmse_px_max_ok	inlier_ratio	match_medi	t_ms
loftr	none	80	42.5	0.208	0.963	0.619	1909	3954
orb	clahe	80	68.8	0.499	1.0	0.619	2503	116
orb	none	80	67.5	0.438	0.999	0.608	2377	107
orb	sauvola	80	55.0	0.479	0.995	0.486	1945	103
sift	clahe	80	82.5	0.186	0.927	0.752	2289	377
sift	none	80	80.0	0.182	0.969	0.691	1709	323
sift	sauvola	80	73.8	0.206	0.927	0.614	974	371

E1 — RMSE vs degradazione (preprocess: none)
 mediana, banda interquartile; log perché gli errori grossolani sono 10^4 volte i buoni



E1 — tasso di successo
 (RMSE sotto 1 px, cioè 0.25 m)



8.1 Il tetto di prestazione

In assenza di degradazione, **sulle prove che riescono**, tutti e tre i matcher recuperano la trasformazione con un errore nell'ordine del pixel o meno: nel caso peggiore 0.502 px per SIFT (0.128 m), 0.294 per LoFTR, 1.000 per ORB — quest'ultimo esattamente al limite di un pixel, non più sotto. Sul numero di SIFT, che resta il riferimento: su un pavimento di riferimento ~0.5 m questo sta sotto di un fattore quattro, ed è il segnale che la pipeline è corretta.

La riserva conta però quanto l'affermazione, perché non tutte le prove riescono. Su tutte e 35 le combinazioni di ritaglio e trasformazione, sempre senza alcuna degradazione, il caso peggiore è 0.5 px per SIFT ma **1.4 px per ORB e 33128 px per LoFTR**. A degradazione nulla l'unica variabile è la trasformazione geometrica — la carta è identica in tutte le prove — quindi quei fallimenti sono **geometrici e non radiometrici**.

Scomponendo per ampiezza della rotazione si vede esattamente dove, e il quadro è netto (RMSE mediano in pixel, senza preprocessing):

matcher	0°	5°	15°	30°	45°	90°
SIFT	0.000	0.033	0.252	0.181	0.270	0.500
ORB	0.000	0.216	0.891	0.378	0.452	0.446
LoFTR	0.030	0.073	1.100	1.270	1745	8715

Per SIFT e ORB l'errore cresce e oscilla con la rotazione ma resta sotto il pixel fino a 90°: è il costo dell'interpolazione del warp e della quantizzazione dell'orientamento dei descrittori, non un difetto. **LoFTR invece supera già il pixel a 15°, resta dello stesso ordine di grandezza fino a 30° e poi si rompe**: a 45° l'errore mediano è di circa 1745 pixel, a 90° di circa 8715.

È il limite più netto emerso da E1, e ha una spiegazione strutturale. SIFT e ORB stimano un orientamento dominante per ogni keypoint e ruotano il descrittore di conseguenza: l'invarianza alla rotazione è costruita dentro il metodo, e vale per qualunque angolo. LoFTR non ha keypoint e non ha orientamenti — mette in corrispondenza due griglie dense di feature — quindi quell'invarianza può solo averla imparata dagli esempi di addestramento, che sono fotografie di scene naturali dove le rotazioni oltre i 30° sono rare. Fuori da quella distribuzione non c'è niente che la garantisca, e infatti non la garantisce.

Vale la pena notare che su E2 la rotazione fra storico e vettoriale è modesta, ed è per questo che LoFTR li compete: il suo punto debole non viene sollecitato. Su un foglio scansionato di traverso, invece, sarebbe il primo a cedere.

8.2 La rottura è un precipizio, non una discesa

La scala di degradazione è stata estesa oltre il valore nominale 1.0 proprio perché a quel livello nulla si rompeva ancora. Il comportamento reale è netto (SIFT, senza preprocessing):

degradazione	RMSE mediano	successo	corrispondenze
0.00	0.119 px	100%	2805
0.75	0.202 px	100%	749
1.00	0.409 px	60%	349
1.30	0.969 px	60%	43
1.40	579.839 px	0%	18

Con passi di 0.2 fermi a 1.0, la curva avrebbe mostrato un degrado dolce che non esiste. Il campionamento è stato infittito sopra 1.0 per catturare la soglia: l'errore resta sotto il pixel fino a 1.30 (60% di successo) e poi esplode a quasi 580 px in un solo passo, un precipizio ancora più netto di quanto suggerisse la prima esecuzione.

8.3 Nota sull'aggregazione

Oltre la soglia di rottura RANSAC **restituisce comunque una H**, ma sbagliata di migliaia di pixel; lo stesso capita a LoFTR sulle rotazioni ampie di §8.1, dove nessuna degradazione è in gioco. Il massimo osservato su E1 è 33128 px. Una sola stima di questo tipo trascina la media dell'intero gruppo, producendo "RMSE medi" di centinaia di pixel che non descrivono né i casi buoni né i cattivi.

È la ragione della convenzione dichiarata in §5.1: le tabelle riportano la **mediana**, che un singolo valore enorme non sposta, accompagnata dal **tasso di successo**, che è la grandezza incaricata di descrivere i casi cattivi. Una media li mescolerebbe entrambi in un numero che non descrive né gli uni né gli altri.

9. E2 — cross-domain reale

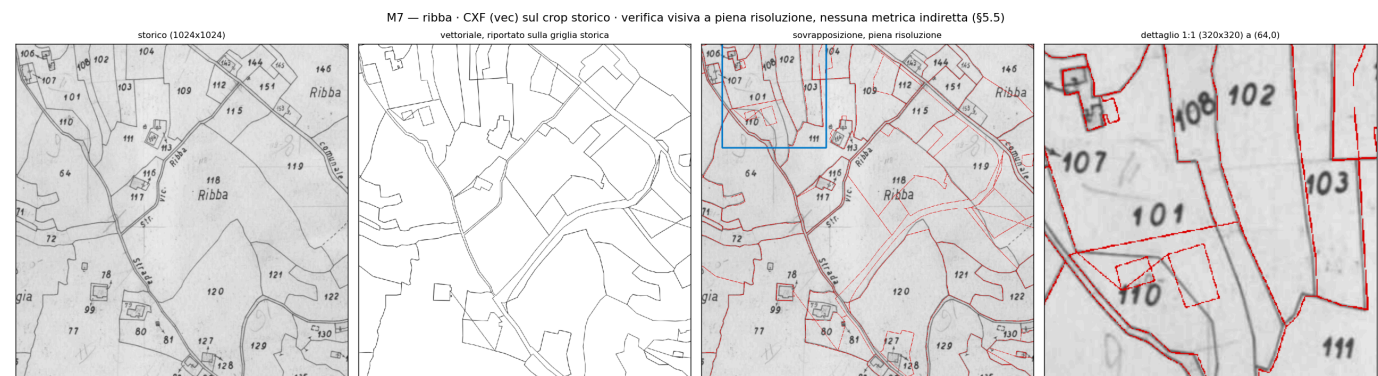
Il secondo esperimento è quello vero: ritaglio storico contro **raster del CXF**, nello stesso sistema di riferimento.

9.1 La rasterizzazione e la sua verifica

I poligoni del CXF sono disegnati come **polilinee e non come campiture**: sul foglio d'impianto il confine è un tratto, e riempire i poligoni cambierebbe il tipo di struttura che il matcher vede.

Una scelta merita di essere motivata: **la griglia del raster vettoriale non è quella del ritaglio storico**. Ha risoluzione propria (0.20 m/px contro 0.254453) e un margine di 20 m. Se le due griglie coincidessero, **H_true** sarebbe l'identità, e l'esperimento premierebbe qualunque metodo con un bias verso il "non muovere niente" — un risultato che sembrerebbe ottimo e non significherebbe nulla. Con griglie diverse **H_true** è una similarità vera, con fattore di scala 1.272265.

La verifica dell'allineamento è **visiva e a piena risoluzione**, non metrica.



Il motivo di questa scelta è documentato in §12.2: su questi dati le metriche indirette di allineamento producono falsi positivi convincenti.

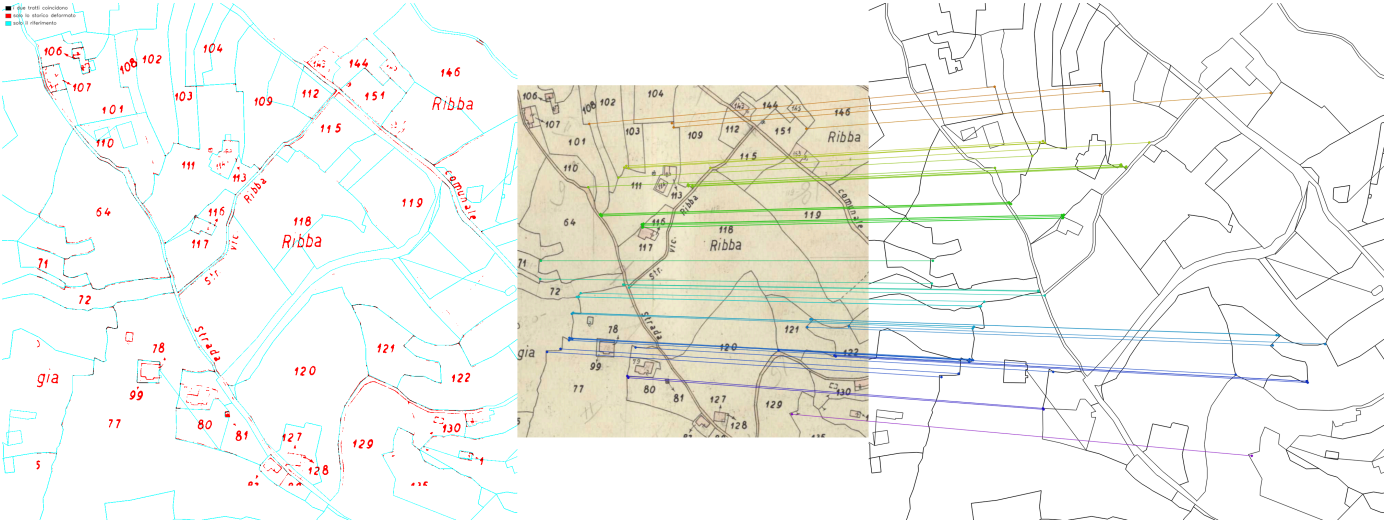
9.2 Risultati

matcher	preprocess	modello	prove	successo_pct	rmse_m_mediano	rmse_m_minimo	inlier_ratio	match_mediani
loftr	clahe	affine	10	0.0	568.2	98.409	0.464	6
loftr	clahe	homography	10	0.0	255.6	119.479	0.619	6
loftr	clahe	similarity	10	0.0	189.12	133.99	0.31	6
loftr	sauvola	affine	10	80.0	0.6	0.35	0.275	385
loftr	sauvola	homography	10	70.0	0.8	0.491	0.273	385
loftr	sauvola	similarity	10	90.0	0.35	0.134	0.288	385
loftr	sauvola+chiusura	affine	10	80.0	0.49	0.348	0.288	390
loftr	sauvola+chiusura	homography	10	80.0	0.91	0.371	0.279	390
loftr	sauvola+chiusura	similarity	10	90.0	0.42	0.257	0.278	390
orb	clahe	affine	10	0.0	83.17	20.138	0.009	777
orb	clahe	homography	10	0.0	132.11	2.453	0.012	777
orb	clahe	similarity	10	60.0	0.65	0.21	0.016	777
orb	sauvola	affine	10	70.0	1.17	0.246	0.034	752
orb	sauvola	homography	10	50.0	2.33	0.555	0.022	752
orb	sauvola	similarity	10	90.0	0.38	0.222	0.049	752
orb	sauvola+chiusura	affine	10	50.0	2.76	0.225	0.032	743
orb	sauvola+chiusura	homography	10	30.0	13.4	0.715	0.022	743
orb	sauvola+chiusura	similarity	10	90.0	0.33	0.165	0.049	743
sift	clahe	affine	10	0.0	212.1	140.125	0.09	82
sift	clahe	homography	10	0.0	133.6	109.56	0.243	82
sift	clahe	similarity	10	0.0	133.96	108.773	0.268	82
sift	sauvola	affine	10	40.0	66.83	0.238	0.07	146
sift	sauvola	homography	10	10.0	131.38	0.31	0.13	146
sift	sauvola	similarity	10	20.0	128.39	0.179	0.105	146
sift	sauvola+chiusura	affine	10	10.0	108.57	0.959	0.078	130
sift	sauvola+chiusura	homography	10	0.0	122.86	7.008	0.117	130
sift	sauvola+chiusura	similarity	10	30.0	118.73	0.322	0.084	130

Sulle 180 prove classiche — SIFT e ORB; le 90 righe LoFTR della tabella sono taggate anch'esse E2 nel CSV, ma si commentano a parte in §10 — 55 raggiungono

un RMSE sotto i 2 m.
Il cross-domain non fallisce del tutto, ma il quadro ribalta E1 su ogni asse.

La migliore combinazione è ORB + Sauvola con chiusura + similarità: 90% di successo, RMSE mediano 0.33 m. È sotto il pavimento del riferimento: la registrazione è buona quanto questa ground truth consente di misurare. Anche Sauvola senza chiusura raggiunge il 90%, con errore mediano 0.38 m: a decidere non è la chiusura, è la coppia binarizzazione più modello vincolato.



9.3 Perché ORB batte SIFT, contro ogni aspettativa

Su E1 SIFT domina; su E2 crolla al 40% di successo mentre ORB arriva al 90%. La causa non è la qualità dei descrittori ma il **numero di candidati**: il ratio test di Lowe lascia a SIFT 93-189 corrispondenze, il cross-check di ORB ne lascia circa 730. Con inlier ratio dell'1-5%, RANSAC ha bisogno di candidati, non di candidati puliti.

L'ipotesi naturale è che il ratio 0.75, tarato su immagini dello stesso dominio, sia semplicemente troppo severo. **È stata verificata, ed è falsa:**

ratio	match_mediani	inlier_mediani	inlier_ratio	rmse_m_mediano	riuscite
0.75	168	11	0.1089	135.4	1/5
0.85	544	43	0.0773	135.4	0/5
0.95	1764	106	0.0641	128.6	0/5
0.99	3191	155	0.057	128.8	0/5

Allentando il ratio fino a 0.99 le corrispondenze passano da ~100 a ~2500, ma l'RMSE resta a centinaia di metri, e su un ritaglio peggiora addirittura da 0.50 m a 129 m. **I match aggiuntivi non contengono segnale:** sono rumore che sposta il consenso di RANSAC su un modello sbagliato. Il limite di SIFT su questi dati è nei descrittori, non nel filtro che li seleziona — che è una conclusione diversa, e più forte.

9.4 Il fattore che decide: il modello geometrico

fattore	valore	prove	successo_pct	rmse_m_mediano	inlier_ratio
modello geometrico	affine	90	36.7	28.04	0.076
modello geometrico	homography	90	26.7	109.17	0.135
modello geometrico	similarity	90	52.2	0.88	0.11
codici CXF	18	135	34.8	50.73	0.106
codici CXF	18+12	135	42.2	28.04	0.104

Sugli stessi identici insiemi di corrispondenze, il modello geometrico cambia tutto: la **similarità** (4 gradi di libertà) riesce nel 52% delle prove, l'affine nel 37%, l'**omografia** (8 gradi) nel 27%. Con inlier ratio bassi — sotto il 5% per ORB, attorno al 10% per SIFT — più gradi di libertà significano più modi di accordarsi con gli outlier: RANSAC trova un consenso, ma quello sbagliato.

E su questi dati la famiglia giusta **si conosce**: §9.1 stabilisce che, avendo le due griglie risoluzioni diverse, **H_true** è una similarità vera con fattore di scala 1.272265. I 2 gradi di libertà in più dell'affine e i 4 dell'omografia non servono a rappresentarla. La tabella non sta quindi misurando quale modello descriva meglio i dati, ma **il costo di concedere allo stimatore più libertà di quanta la verità ne richieda**: 26 punti di tasso di successo per l'omografia. Si potrebbe obiettare che la deformazione residua della carta giustifichi i gradi di libertà dell'affine, ma la misura dice che non li ripaga — l'affine perde 16 punti invece di guadagnarne.

È il risultato più trasferibile dell'intero lavoro: **su dati cross-domain con inlier ratio bassi, il modello più vincolato non è una semplificazione, è una necessità**.

La tabella mostra anche l'esito dell'ablazione sui codici CFX: rasterizzare **particelle + acque/strade** (18+12) batte le sole particelle (18), 42% contro 35% di successo. Le strade e i corsi d'acqua aggiungono struttura proprio dove il tratto storico è più marcato.

9.5 I ritagli non sono equivalenti

crop	codici	n_matches	inlier_ratio	rmse_m	success
aspera	18	664	0.009036	78.296	False
aspera	18+12	766	0.011749	0.516	True
cannei	18	727	0.060523	0.372	True
cannei	18+12	836	0.077751	0.222	True
ribba	18	739	0.071719	0.38	True
ribba	18+12	856	0.060748	0.396	True
tassarole	18	671	0.04769	1.252	True
tassarole	18+12	792	0.04798	0.371	True
vedra	18	613	0.050571	0.318	True
vedra	18+12	808	0.048267	0.312	True

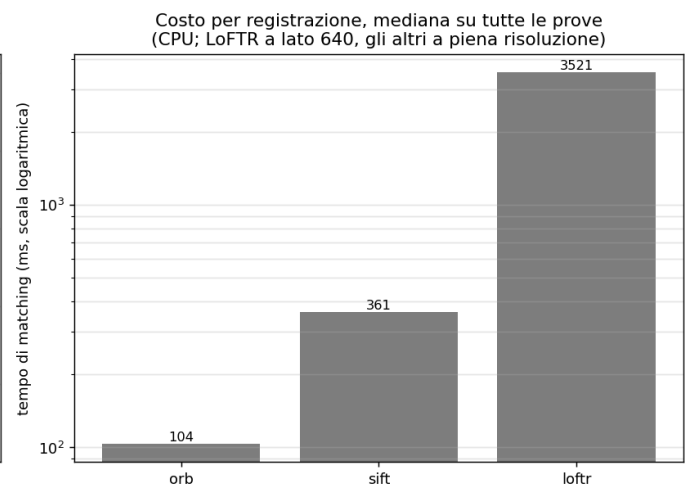
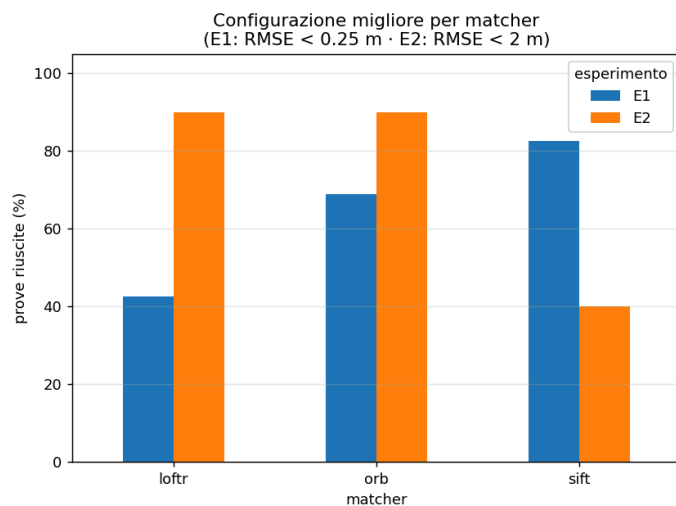
Un caso è istruttivo: **aspera fallisce con le sole particelle (78.3 m) e riesce includendo acque e strade (0.52 m)**. È il ritaglio che tocca la costa, dove buona parte del contenuto sono la linea di riva e i corsi d'acqua: senza il codice 12 il vettoriale è quasi vuoto proprio dove l'impianto ha il tratto. La stessa configurazione ha anche l'inlier ratio più basso fra tutte le prove riuscite (0.012, contro il 5-8% degli altri crop): la registrazione tiene anche quando solo l'1% circa delle corrispondenze è corretto.

10. E3 — classico contro neurale

LoFTR entra nella pipeline dalla stessa porta degli altri: cambia solo il valore di **--matcher**. Stessi ritagli, stesse metriche, stesse soglie.

esperimento	matcher	config	prove	successo_pct	rmse_m_mediano_ok	inlier_ratio	match_mediani	t_ms
E1	loftr	none / homography	80	42.5	0.053	0.622	1033	3875
E1	orb	clahe / homography	80	68.8	0.127	0.705	2511	114

esperimento	matcher	config	prove	successo_pct	rmse_m_mediano_ok	inlier_ratio	match_mediani	t_ms
E1	sift	clahe / homography	80	82.5	0.047	0.873	1497	378
E2	loftr	sauvola / similarity	10	90.0	0.342	0.288	385	3238
E2	orb	sauvola+chiusura / similarity	10	90.0	0.284	0.049	743	87
E2	sift	sauvola / affine	10	40.0	0.861	0.07	146	362



10.1 Due differenze da dichiarare

Il confronto è onesto solo se si dichiara ciò che non è simmetrico:

1. **LoFTR gira a lato massimo 640 px**, mentre SIFT e ORB girano a piena risoluzione. LoFTR lavora su una griglia a 1/8 e il costo cresce col quadrato del lato: a 1024 px su CPU diventa proibitivo. I keypoint vengono riportati alle coordinate originali, quindi **H_{est}** resta nei pixel di partenza.
2. **Il costo per registrazione è di un altro ordine di grandezza**, e la colonna **t_{ms}** lo riporta per la configurazione migliore di ciascun matcher. Fa parte del risultato. Quel tempo misura la sola inferenza: il caricamento dei pesi di LoFTR — 90 MB — è cronometrato a parte (§5.1), perché è un costo di natura diversa. Si paga una volta e si ammortizza già alla seconda registrazione; quello ricorrente è ciò che conta quando i fogli da registrare sono molti.

10.2 LoFTR non ribalta il cross-domain

Sul tasso di successo LoFTR **pareggia** ORB (90%), con RMSE mediano peggiore (0.342 contro 0.284 m) e un tempo per registrazione di un ordine di grandezza superiore, che si legge nella colonna **t_{ms}**. La promessa del detector-free — funzionare dove i rilevatori a blob non hanno nulla da agganciare — **non si realizza su questi dati**.

Dove è invece nettamente superiore è nella **qualità** delle corrispondenze. Il confronto va però fatto sulle configurazioni che registrano davvero, e per una ragione che vale la pena esplicitare: le tre configurazioni LoFTR con CLAHE hanno gli inlier ratio più alti dell'intera griglia (0.464, 0.619, 0.31) e insieme sei corrispondenze mediane e un tasso di successo dello 0%. Un inlier ratio calcolato su sei corrispondenze non è confrontabile con uno calcolato su settecento: mediare sull'intera griglia **premia proprio le celle in cui il matcher non ha trovato niente**.

Restringendo alle righe con Sauvola — quelle in cui tutti e tre i matcher registrano — il quadro è netto. Gli intervalli coprono i tre modelli geometrici e le due varianti di binarizzazione, e si leggono dalla tabella di §9.2:

matcher	inlier ratio	corrispondenze mediane
LoFTR	0.273 – 0.288	385 – 390
SIFT	0.070 – 0.130	130 – 146
ORB	0.022 – 0.049	743 – 752

Sono due strade opposte allo stesso risultato: LoFTR trova un numero moderato di corrispondenze quasi tutte utilizzabili, ORB ne trova una massa in cui gli inlier stanno sotto il 5% e lascia a RANSAC il lavoro di setacciarle. Il solo tasso di successo nasconde questa differenza.

10.3 Anche LoFTR ha bisogno della binarizzazione

preprocessing	corrispondenze mediane	successo	RMSE mediano
CLAHE	6	0%	189 – 568 m
Sauvola	385	70 – 90%	0.35 – 0.80 m
Sauvola + chiusura	390	80 – 90%	0.42 – 0.91 m

Il vantaggio del pre-addestramento su immagini naturali **non sopravvive al divario di dominio**: a colmarlo è la binarizzazione, non la rete. È forse il risultato più interessante della componente comparativa, perché suggerisce che su questo tipo di dati la leva efficace resti il preprocessing.

10.4 Fragilità alla degradazione

Su E1, tasso di successo al crescere della degradazione (senza preprocessing):

degradazione	LoFTR	ORB	SIFT
0.00	100%	80%	100%
0.50	100%	80%	100%
1.00	40%	40%	60%
1.10	0%	40%	80%
1.40	0%	0%	0%

LoFTR parte alla pari con SIFT — e sopra ORB — e **crolla prima di entrambi i classici**: a degradazione 1.1 è già a zero, dove SIFT tiene l'80% e ORB il 40%. Il rumore gaussiano non appartiene alla distribuzione su cui è stato addestrato.

La colonna di ORB non è monotona, e non va letta come se lo fosse: ogni cella aggrega **cinque prove**, una per ritaglio, quindi un solo successo in più o in meno vale venti punti percentuali. È il rumore di campionamento dichiarato in §12.4, non un fenomeno da raccontare.

11. Modalità d'uso

11.1 Installazione

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Per il solo approccio classico bastano `numpy`, `opencv-python`, `Pillow`, `scipy`, `pandas`, `matplotlib`. `torch` e `kornia` servono **solo a** `--matcher loftr`; su CPU conviene l'indice dedicato:

```

pip install torch==2.3.1 --index-url https://download.pytorch.org/whl/cpu
pip install kornia==0.7.3
python -m scripts.scarica_pesì # pesi LoFTR + verifica del checksum

```

I dati cartografici non sono versionati: [data/README.md](#) documenta come ricostruirli.

11.2 La CLI

```

python -m src.main --hist <crop.png> --modern <raster.png> [opzioni]

--hist <path>          immagine storica (obbligatorio)
--modern <path>        immagine di riferimento (obbligatorio)
--matcher <sift|orb|loftr> default: sift
--preprocess <none|clahe|otsu|sauvola> default: sauvola
--morph-close <n>      default 0
--morph-open <n>       default 0
--model <similarity|affine|homography> default: homography
--ratio <float>        ratio test di Lowe, default 0.75
--ransac-thresh <float> px, default 3.0
--seed <int>           default 42
--jgw-hist <path>       world file storico (solo evaluate)
--jgw-modern <path>     world file moderno (solo evaluate)
--soglia-m <float>      soglia di successo in metri
--out-csv <path>        default results/runs.csv (in append)
--out-figure <path>     overlay del warp + corrispondenze
--esperimento <str>     etichetta nella colonna `esperimento` del CSV
--crop <str>            etichetta nella colonna `crop` del CSV
--verbose

```

Esempio 1 — la configurazione migliore su dati reali:

```

python -m src.main \
  --hist data/crops/ribba.png --modern data/crops/ribba_vec.png \
  --matcher orb --preprocess sauvola --model similarity \
  --jgw-hist data/crops/ribba.jgw --jgw-modern data/crops/ribba_vec.jgw \
  --soglia-m 2.0 --out-figure results/figures/registrazione.png

```

Esempio 2 — senza world file: la pipeline gira lo stesso. È la prova che la georeferenziazione non entra nell'algoritmo:

```

python -m src.main --hist data/crops/ribba.png --modern data/crops/ribba_vec.png \
  --matcher sift --preprocess sauvola
# → H_est prodotta, "nessun world file: RMSE non calcolato"

```

Esempio 3 — lo stesso confronto con il matcher neurale, cambiando una parola:

```

python -m src.main --hist data/crops/ribba.png --modern data/crops/ribba_vec.png \
  --matcher loftr --preprocess sauvola --model similarity

```

11.3 Le milestone, e i nomi dei file

Lo sviluppo è stato diviso in dieci milestone, ciascuna con un programma che gira e produce un artefatto guardabile. **Il numero sopravvive nei nomi dei file**, e serve a leggerli: [m6_e1_completo.py](#) è l'esperimento della milestone 6, [m8_verifica_ribba.png](#) una figura della milestone 8. La tabella è quindi anche la chiave dei comandi e delle figure che seguono.

#	che cosa produce	dove sta in questo documento
M1	lettura di JPG e world file, i cinque ritagli	\$3.4
M2	il parser CXF, con l'assert sull'estensione	\$3.3
M3	la composizione della trasformazione di riferimento	\$4
M4	la prima coppia sintetica con H nota: se qui l'errore non è ≈ 0 , il problema è nel codice e si ferma tutto	\$8
M5	il confronto dei preprocessing sui ritagli reali	\$6
M6	E1 completo, la curva RMSE contro degradazione e il primo CSV	\$8
M7	la rasterizzazione del vettoriale, con verifica visiva a piena risoluzione	\$9.1
M8	E2, la griglia cross-domain intera	\$9
M9	E3, LoFTR nella stessa pipeline	\$10
M10	le figure e le tabelle generate dal CSV, e questo documento	\$5.1, \$11.5

Le milestone non sono fasi di un piano dichiarato a posteriori: erano il criterio per andare avanti. Ognuna doveva produrre qualcosa di ispezionabile prima che si passasse alla successiva, e M4 in particolare era una condizione d'arresto — senza un errore quasi nullo sul sintetico, misurare il cross-domain non avrebbe significato niente.

11.4 Preparazione dei dati ed esperimenti

```
# ritagli dal foglio (PNG + world file affiancato)
python -m src.prep.crop

# ispezione e verifica del parser CXF
python -m src.io_geo

# rasterizzazione del vettoriale sulla zona di un ritaglio
python -m src.prep.rasterize --crop ribba --codici 18,12
python -m experiments.m7_rasterize_check --crop ribba

# confronto dei preprocessing
python -m experiments.m5_preprocess --crop tutti --dettaglio ribba

# gli esperimenti: E1, E2, la diagnosi del ratio, E3
python -m experiments.m6_e1_completo --riparti
python -m experiments.m8_e2_griglia
python -m experiments.m8_e2_griglia --diagnosi-ratio
python -m experiments.m9_e3_loftr

# figure e tabelle, tutte generate dal CSV
python -m src.report --csv results/runs.csv

# questa relazione, con le tabelle iniettate dal CSV
python -m scripts.componi_relazione

# i test
python -m tests.test_smoke
```

11.5 Riprodurre tutto in un comando

I comandi di §11.4 vanno eseguiti in quest'ordine, e l'ordine non è arbitrario: i ritagli prima della rasterizzazione, la rasterizzazione prima di E2, gli esperimenti prima delle tabelle, le tabelle prima di questo documento. Eseguirli a mano funziona, ma un passo dimenticato non dà errore: produce un CSV parziale e tabelle che sembrano complete.

```
python -m scripts.riproduci --controlla # verifica le precondizioni, non esegue
python -m scripts.riproduci --lista    # le fasi, in ordine, con i tempi
python -m scripts.riproduci            # tutto tranne E3 (~18 min)
python -m scripts.riproduci --con-loftr # tutto, E3 compreso (~60 min)
python -m scripts.riproduci --da-verifica-raster # senza i dati AdE, dai soli ritagli
```

Lo script stampa ogni comando prima di eseguirlo — il log di una corsa è la versione eseguita di \$11.4 — e dopo ogni fase verifica che gli artefatti attesi esistano davvero: un comando che esce con codice 0 senza aver scritto quello che doveva ferma la corsa, che riprende con `--da <fase>`. In coda controlla che ogni figura citata da questa relazione sia stata prodotta e che nessun segnaposto di tabella sia rimasto vuoto.

Il controllo delle precondizioni non è formalità. Gli esperimenti di E1 ed E3 convertono l'errore in metri leggendo la risoluzione da un world file, e se nessuno è disponibile l'errore in metri resta indefinito per ogni riga: l'esperimento gira fino in fondo e conclude "0 riuscite". Si legge come un algoritmo che fallisce, ed è invece un file assente, e la difesa è verificare prima.

Il controllo è però **per fase**, e la distinzione conta proprio alla consegna. Le scansioni catastali non sono ridistribuibili, i ritagli sì: chi riceve il progetto senza `data/raw/` ha comunque in `data/crops/` i ritagli, i raster del vettoriale e i rispettivi world file. Solo `crop`, `cxr` e `rasterize` aprono le scansioni; le altre nove fasi no.

12. Limiti e onestà dei risultati

12.1 Il vettoriale non è la digitalizzazione dell'impianto

Il CXF è la **cartografia vigente**, non una trascrizione del foglio storico. La geometria discende dall'impianto, ma ha subito un secolo di frazionamenti e aggiornamenti: nel CXF compaiono particelle con numeri a quattro cifre (1015, 1026, 1041) che sull'impianto non esistono, dove la numerazione arriva a ~336.

Parte delle linee vettoriali non ha alcuna corrispondenza nell'inchiostrato, e si vede a occhio nelle sovrapposizioni a piena risoluzione. Non è disallineamento: è storia. È anche una delle ragioni per cui gli inlier ratio di E2 sono strutturalmente bassi, e vanno letti con questa chiave.

12.2 Le metriche indirette non funzionano su questi dati

Durante la preparazione si è tentato di misurare l'allineamento con correlazione incrociata e chamfer matching su distance transform. **Entrambe hanno prodotto falsi positivi convincenti**: shift ottimali incoerenti fra zone (−238, −330, +11 px), ottimi saturati sui bordi dello spazio di ricerca, e un profilo apparentemente sistematico che confermava un disallineamento inesistente.

La diagnosi è quantitativa: **linee tirate a caso sul foglio ottengono l'11.5% dei pixel entro 2 px dall'inchiostrato; il vettoriale vero il 16-20%**. Con quel rapporto segnale/rumore nessuna di quelle metriche discrimina — la carta ha inchiostrato sparso ovunque (testi, simboli, tratteggi, grana, macchie) e la superficie di correlazione su disegni al tratto è piatta e multi-picco.

Da qui la regola seguita in tutto il progetto: **l'unica metrica di valutazione è l'RMSE su checkpoint contro `H_true`**; ogni operazione di allineamento si verifica a piena risoluzione; ogni misura indiretta va accompagnata da una baseline casuale, e se il segnale non batte nettamente il caso, la misura si butta.

12.3 Validazione, non applicazione

Questo va detto prima che lo chieda chi legge. I dati dell'Agenzia delle Entrate forniscono una georeferenziazione **già nota**, e *proprio per questo* permettono di valutare quantitativamente un metodo che altrove — scansioni

d'archivio, catasti preunitari, mappe senza world file — dovrebbe operare senza alcun riferimento.

È un dataset di **validazione**, non di applicazione. Scrivere che il lavoro "serve ad allineare mappe storiche disorientate" sarebbe falso *su questi dati*: qui il riferimento c'è, ed è quello che rende possibile la misura.

12.4 Ampiezza del campione

Ogni punto delle curve di E1 è la mediana di 5 prove, una per ritaglio; ogni cella delle tabelle di E2 aggrega 10 prove. Le non-monotonie che compaiono qua e là — per esempio nel tasso di successo attorno a degradazione 1.1 — sono rumore di campionamento, non struttura, e non vanno raccontate come fenomeni.

12.5 Un errore documentato nel materiale di partenza

La specifica del progetto riportava il foglio come 8000×5322 px, con l'estensione corrispondente. Il file reale misura **8489×5648 px**: i pixel in più sono il margine di carta della scansione. L'ipotesi alternativa — che il world file appartenesse a un rendering diverso, e che quindi il pixel valesse 0.2398 m anziché 0.254453 — avrebbe comportato un errore di scala del 6% su *tutti* i risultati in metri.

È stata risolta con una verifica visiva a piena risoluzione: con il JGW applicato al file reale il vettoriale ricalca l'inchiestro, mentre un errore del 6% significherebbe circa 500 px di divergenza sul foglio. Il codice legge sempre la dimensione dal file, mai da una costante.

13. Conclusioni

Il lavoro misura, e spiega, dove i metodi classici cedono sul cross-domain — e dove invece reggono meglio del previsto.

1. **La pipeline è corretta**: E1 recupera trasformazioni note con errore sub-pixel (0.046 m mediani nella configurazione migliore), circa un ordine di grandezza sotto il pavimento del riferimento.
2. **La registrazione cross-domain riesce**, ma non con la configurazione che ci si aspetterebbe: ORB + Sauvola con chiusura + similarità raggiunge il 90% di successo con RMSE mediano 0.33 m, al limite di ciò che questa ground truth può misurare.
3. **Il modello geometrico conta più del matcher**: a parità di corrispondenze, passare da omografia a similarità porta il successo dal 27% al 52%. Con inlier ratio bassi, vincolare è necessario.
4. **Il preprocessing conta più della rete**: su E2 è la binarizzazione di Sauvola a far funzionare tutti e tre i matcher, LoFTR incluso. Con CLAHE SIFT non riesce mai e LoFTR quasi mai; ORB tiene solo se vincolato alla similarità (60% di successo) e va a zero con affine e omografia.
5. **Il matcher neurale non ribalta il risultato**: pareggia ORB sul successo, con errore maggiore e un costo per registrazione di un ordine di grandezza superiore — ma con corrispondenze molto più pulite, il che indica che il collo di bottiglia è altrove. Ha però un limite che i classici non hanno: **oltre i 30° di rotazione si rompe** (§8.1), perché l'invarianza che SIFT e ORB costruiscono per progetto lui può solo averla imparata dai dati di addestramento. Su questi ritagli non si vede, perché la rotazione in gioco è piccola; su un foglio scansionato di traverso sarebbe il primo a cedere.
6. **Due ipotesi sono state smentite dai dati** e riportate come tali: il ratio test troppo severo (§9.3) e il fallimento atteso di Otsu (§6.3).

Un progetto che misura e spiega perché un metodo cede vale più di uno che mostra solo il caso riuscito. Qui i casi riusciti ci sono, e i punti di rottura sono documentati con lo stesso rigore.

Resta la domanda di che cosa, di tutto questo, valga fuori dal foglio 49. Non i numeri: quelli descrivono questa carta, questa scala, questo vettoriale. Valgono

le decisioni che li hanno prodotti — vincolare il modello geometrico quando l'inlier ratio è basso, ripulire il segnale con una soglia locale invece che globale, non credere a una misura indiretta che non batta nettamente una baseline casuale — e valgono perché sono state scelte confrontando alternative contro una verità esatta, non per plausibilità. È a questo che serve un insieme di validazione: decidere dove il riferimento c'è, per poter lavorare dove non c'è. Su una scansione d'archivio priva di world file quelle decisioni si trasferiscono immutate; quello che non si trasferisce è il criterio di accettazione, perché lì l'RMSE in metri non è calcolabile, e al suo posto resta l'ispezione a piena risoluzione — più lenta, meno conclusiva, e l'unica disponibile.